FestOS – Antigravity Build Playbook

24 sequential, copy-paste prompts for a 3-person team (P1 → P2 → P3 rotation)

This playbook splits the full FestOS build (EPINOIA 2026, Aarohan fest) into 24 prompts. Person 1 runs prompts 1, 4, 7, …, 22; Person 2 runs 2, 5, 8, …, 23; Person 3 runs 3, 6, 9, …, 24. Every prompt reads the same shared context, writes only inside its owner's folders, and plugs into the others through registries and a fixed API contract, so nobody overwrites anybody.

0. How to use this playbook

Roles

Person	Role	Prompts
P1	Platform and planning-engines lead: foundation, participants and reconciliation, scheduling, inventory and merchandise, staff and transport, dashboard and simulator, AI commands and reports, final integration	1, 4, 7, 10, 13, 16, 19, 22
P2	Allocation and operations-engines lead: import and assumptions, accommodation, vendors and finance, impact and disruption engine, burn-down, pre-mortem and forecasting, hardening, submission docs	2, 5, 8, 11, 14, 17, 20, 23
P3	Security and frontend lead: auth, RBAC, audit, approvals, design system, every UI screen, PWA, E2E tests, pitch	3, 6, 9, 12, 15, 18, 21, 24

Step 0 (all three, about 15 minutes, before any prompt)

1. P1 creates the Git repo, creates `docs/FESTOS_CONTEXT.md` by pasting Section A below verbatim, copies the data pack CSVs into `data/raw/` and the problem-statement PDF plus master specification into `docs/spec/`, then pushes `main` and `develop`.
2. P2 and P3 clone, install Python 3.11, Node 20+, Docker, and open the repo as the Antigravity workspace.

3. If your Antigravity version supports workspace rules, save Section A as a rule so every agent session loads it. Otherwise each prompt already tells the agent to read `docs/FESTOS_CONTEXT.md` first.

Order is ownership order, not a lock. Start your next prompt as soon as the prompts it lists under Depends on are merged into `develop`. If a dependency has not landed, code against its contract in `docs/api/<module>.md` and mocks, then rebase when it merges. P2 and P3 can begin Prompts 2 and 3 on pure logic and UI while P1 finishes Prompt 1, but must rebase on P1's interface-first push (core, models, schemas) which lands in roughly the first 30 minutes.

Ritual for every prompt

1. `git checkout develop && git pull`, then `git checkout -b pN/NN-slug`.
2. Ask the agent to produce a plan first and check it against Section A and the prompt's file list before it writes code.
3. Paste the prompt.
4. Run `make lint test`, regenerate types with `make gen-types`.
5. Push the contract (`docs/api/<module>.md` plus schema stubs) early; push the rest when green.
6. Open a PR into `develop` using the checklist in Section A.12, add a line to `docs/INTEGRATION_LOG.md`, and post a short message to the team: endpoints, `public.py` functions, anything breaking.

Honest caveats

- I have not seen the CSV files, only the problem statement and the master specification. Prompt 2 therefore starts by profiling the real files, and all later prompts depend on the canonical schema, never on raw column names.
- Numbers such as 1,601 peak beds or 987 unscheduled registrations come from an earlier prototype described in the specification. Tests use them as sanity checks and documented reference points, not targets to hardcode.
- Antigravity-specific UI features vary by version, so prompts are written as plain instructions that work in any agent chat.

A. FESTOS SHARED CONTEXT (save as `docs/FESTOS_CONTEXT.md`)

```
FESTOS SHARED CONTEXT v1.0. Every agent session reads this first. Change it
only through a PR labelled `context`.

1. PRODUCT PRINCIPLES
- FestOS is the internal command centre for the Aarohan fest (EPINOIA 2026,
IIM Indore). It is not a registration site and not a CRUD portal. Scale:
~2,400 expected vs 2,836 raw participants (never force 2,400: reconcile and
show the gap), 38 contingents, 31 events, 9 venues, 14 vendors, 48 core team,
200 rooms / 710 beds on campus, 6 external properties / 740 beds.
- A change is complete only when its downstream effects are visible.
- AI proposes. Deterministic logic validates. Humans approve.
- UNKNOWN is a first-class value. A blank never means NO, consent, a gender,
dietary clearance or absence of need.
- Hard constraints (accessibility, privacy, safety, required housing
preference) are never auto-relaxed. Soft constraints are relaxed only with a
recorded reason, in this order: cohesion, cost, proximity.
- Every mutation goes through ChangeService. Every important decision has a
Why card. Every high-risk override lands in the override register.
- Assumptions are visible, editable, owned and sourced. Heuristics are never
called optimal. Schedules generated by FestOS are labelled ASSUMED. Synthetic
data only.
- Reference numbers in the spec (1,601 peak beds, 987 unscheduled
registrations, 71 minors external, ...) come from an earlier prototype. Use
them as sanity checks, never hardcode them.

2. STACK (modular monolith, no microservices)
Backend: Python 3.11, FastAPI, SQLModel/SQLAlchemy 2, SQLite in WAL mode by
default (DATABASE_URL may point to Postgres), Pandas, RapidFuzz, OR-Tools
(optional), pytest + hypothesis, ruff, black.
Frontend: React 18 + Vite + TypeScript strict, Tailwind, React Router,
TanStack Query, Recharts, openapi-typescript, MSW, vitest, Playwright, vite-
plugin-pwa.
Ops: Docker Compose, Makefile, GitHub Actions.

3. REPO LAYOUT
festos/
  Makefile  docker-compose.yml  .env.example  CODEOWNERS
  data/raw  data/mapping  data/golden  data/snapshots
  docs/  FESTOS_CONTEXT.md  spec/  api/<module>.md  DATA_PROFILE.md
```

```
DECISIONS.md  INTEGRATION_LOG.md  TEST_CREDENTIALS.md
  scripts/
  backend/requirements.txt
  backend/app/
    main.py
    core/      config db errors clock scoring bus registry idempotency
change_service | audit security rbac masking deps
    schemas/   common.py  <module>.py
    models/    participants events inventory staff transport change |
accommodation vendors ops | audit
    api/v1/    <module>.py   (each exports `router`; main.py auto-discovers
them)
    services/<module>/  service.py  public.py  handlers.py
    engines/<name>/     pure logic: no DB access, no FastAPI imports
    seed/      registry.py  users.py  <module>.py
    reports/   <report>.py
  backend/tests/<module>/
  frontend/src/  app/ api/ components/ui components/layout features/<module>/
hooks/ lib/ pwa/ types/generated.ts
  frontend/e2e/

OWNERSHIP (never edit a file you do not own except by PR with its owner as
reviewer, label `cross-owner`)
P1: repo root, CI, Docker, scripts,
core/{config,db,errors,clock,scoring,bus,registry,idempotency,change_service}
, schemas/common.py,
models/{participants,events,inventory,staff,transport,change}.py,
seed/registry.py, initial frontend scaffold (Prompt 1 only), and modules:
participants, contingents, reconciliation, events, venues, schedule,
logistics, merchandise, staff, transport, dashboard, decisions, scenarios,
commands, reports.
P2: models/{accommodation,vendors,ops}.py and modules: imports, assumptions,
accommodation, checkin, vendors, food, finance, notifications, incidents,
impact, unknowns, premortem, forecasting.
P3: core/{audit,security,rbac,masking,deps}, models/audit.py, seed/users.py,
modules: auth, audit, approvals, overrides, and the entire frontend/ after
Prompt 1 (including e2e).
Float rule: if P1 or P2 finish early they may build pages in NEW folders
frontend/src/features/<their module>/ after telling P3.

4. CODING STANDARDS
- Python snake_case, classes PascalCase, type hints everywhere, ruff + black.
Routers call services, services call engines. No business logic in routers.
```

```
Engines are pure and unit-tested.
- JSON keys are snake_case. TypeScript API types are generated (`make gen-types` writes frontend/src/types/generated.ts), never hand-copied. React components PascalCase, hooks useX, one feature folder per module slug.
- Module slugs (used for api file, service dir, feature dir, permission prefix, docs/api file): auth audit approvals overrides imports assumptions participants contingents reconciliation unknowns accommodation checkin events venues schedule logistics merchandise vendors food finance staff transport notifications incidents impact scenarios premortem forecasting dashboard decisions commands reports
- IDs: keep source IDs (C05, VND05, ...) as primary keys where the data pack has them; otherwise prefix + zero-pad. Raw rows are preserved in raw_record; canonical rows keep source_row_id.
- Time: store UTC timezone-aware, display Asia/Kolkata. core.clock.now() is the only source of the current time (supports FESTOS_NOW env and a demo-time endpoint).

5. API CONVENTIONS
- Base path /api/v1/<module>. Success envelope: {data, meta, warnings}. Error envelope: {error: {code, message, details, request_id}}.
- Error codes: VALIDATION_ERROR, NOT_FOUND, FORBIDDEN, UNAUTHENTICATED, VERSION_CONFLICT (409), ALREADY_ISSUED (409), CONSTRAINT_VIOLATION (422, details = Violation list), APPROVAL_REQUIRED (202), RATE_LIMITED, INTERNAL.
- Lists accept page, page_size, sort, q plus filters, and return meta.total.
- Mutating requests: `Idempotency-Key` header (mandatory for scan, issue, check-in, sync and commit endpoints); optimistic concurrency through `expected_version` in the body (409 VERSION_CONFLICT). Bodies may carry `reason`. `dry_run=true` returns the impact preview and writes nothing.
- Sensitive fields (contact, medical, accessibility, room data, guardian data) are masked in serializers by core.masking according to the actor's role.
- Contract first: before implementing a module, push docs/api/<module>.md (endpoints, request/response schemas, example JSON) plus Pydantic schema stubs so the frontend can mock them with MSW.

6. DATA CONVENTIONS
- Tri = YES | NO | UNKNOWN. Categorical fields (gender, diet, housing_preference, accessibility_need, ...) store the literal UNKNOWN when missing, never NULL or empty string. field_verification(entity_type, entity_id, field, status UNKNOWN|CLAIMED|VERIFIED, value, source, verified_by, verified_at, owner) tracks confirmation.
- Mutable tables carry id, version, created_at, updated_at. Never hard-delete operational rows; use status.
```

```
- Canonical tables (spec section 11): participant, contingent, registration,
event, event_requirement, venue, travel_time, room, external_property,
stay_assignment, inventory_item, inventory_movement, merch_claim, vendor,
vendor_assignment, staff, audit_event, change_set, assumption, override,
incident, approval, shift, transport, shuttle_assignment.
- Prompt 1 additionally creates: user, break_glass_grant, accommodation_plan
(plan_id; kind A|B|C|LIVE; status CANDIDATE|APPROVED|LIVE|ARCHIVED),
event_session, schedule_version, conflict, inventory_location, meal_slot,
import_batch, raw_record, quarantine_row, duplicate_candidate, merge_record,
field_verification, notification_draft, scenario, idempotency_key. Owners may
add more tables inside their own model files.
- Hard DB guarantees: unique(canonical_participant_id, item_id) on
merch_claim; no overlapping active stay for the same bed within a plan;
append-only audit_event.

7. CHANGE SERVICE (every mutation uses it)
Flow: validate -> check constraints -> snapshot previous state -> apply in
ONE transaction -> audit_event + change_set -> impact hook -> recommendations
-> approval gate.
- ChangeRequest{action, entity_type, entity_id, payload, reason, actor,
related_incident_id, expected_version, dry_run, idempotency_key}
- ChangeResult{change_set_id, status
APPLIED|PENDING_APPROVAL|REJECTED|DRY_RUN, violations, audit_event_ids,
impact, recommendations, warnings}
- Handlers register with @register_change_handler('<module>.<verb>') and
implement validate(req, session) -> list[Violation]; snapshot(req, session) -
> dict; apply(req, session) -> dict; risk(req) -> LOW|HIGH; optional
revert(change_set, session).
- HIGH risk returns PENDING_APPROVAL unless the actor's role may approve and
a reason is given. Approvals and overrides are owned by P3 and called through
core interfaces approvals.require(...) and overrides.record(...).
- In-process domain events: core.bus.publish('participant.status_changed',
payload); consumers use @subscribe and must be idempotent.

8. MODULE PUBLIC CONTRACT (the integration glue)
Every module exposes services/<module>/public.py with READ-ONLY functions:
- summary(session, now) -> dict of dashboard KPIs
- risks(session, now) -> list[RiskItem]   (evaluated as of `now`, so the pre-
mortem can replay any hour)
- dependents(session, entity_type, entity_id) -> list[EntityRef]   (what sits
downstream of this entity; feeds the impact graph)
Modules import other modules only through public.py and schemas, never their
internals. Dashboard, impact graph, simulator, pre-mortem and reports read
```

through these.
Other registries (core/registry.py): register_participant_section(name, fn) for the participant 360 view; register_seeder(name, order, fn) with `python -m app.seed reset` rebuilding the DB (order: 10 import, 20 users, 30 accommodation resources, 40 inventory, 50 vendors, 60 staff).
Shared schemas (schemas/common.py): Tri, Severity (1 LOW, 2 MEDIUM, 3 HIGH, 4 CRITICAL), Actor, Violation{code, severity, hard, message, entity_refs}, EntityRef{type, id, label}, RiskItem{id, module, kind, title, severity, hours_to_impact, people_affected, uncertainty 0-1, resource_criticality 1-5, entity_refs, action{label, route, api}, explanation}, ImpactReport{root_change, nodes[{type, id, label, severity, people_affected, depth, parent, status}], counts_by_type, recommendations, before_after}, DecisionCard{id, subject, recommendation, hard_satisfied, soft_relaxed[{constraint, reason}], alternatives_rejected[{option, why}], cost_impact, people_affected, approved_by, created_at}, Envelope, Page.

9. SHARED ENUMS AND SCORING
- Roles: FESTIVAL_DIRECTOR, ACCOMMODATION_LEAD, EVENT_LEAD, LOGISTICS_MANAGER, MERCHANDISE_MANAGER, FOOD_OPS_MANAGER, FINANCE_MANAGER, VOLUNTEER, AUDITOR. Permission strings: <module>: <read|write|approve|export|sensitive>.
- Attention score = Severity (1-4) x Urgency x max(people_affected, 1). Urgency by hours_to_impact: <=1h 5, <=4h 4, <=12h 3, <=24h 2, otherwise or None 1. core/scoring.py is the single implementation.
- Participant status: EXPECTED, ARRIVED, CHECKED_IN, CHECKED_OUT, CANCELLED, NO_SHOW, WITHDRAWN, INVALID, MERGED. Plan kinds: A cohesion-first, B cost-first, C balanced.

10. ASSUMPTION KEYS (seeded into the assumption table; each has owner, source, reason, impact-if-changed)
cleaning_buffer_minutes = 180 (prototype value; an earlier concept said 60; one configurable value only)
late_night_start / late_night_end = 22:00 / 06:00
minor_age_threshold = 18
setup_buffer_min / teardown_buffer_min = 15 / 15
venue_travel_min = 15 (per-pair override in travel_time)
round_result_buffer_min = 60
semifinal_ratio = 0.5 ; finalist_ratio = 0.10 ; finalist_min = 8
checkin_buffer_min = 60
expected_participants_narrative = 2400 (shown beside the raw count, never forced)
external_budget_inr = NULL (not supplied; engine runs in budget-unset mode and flags it; the Director sets it)

```
external_property_accessibility = UNVERIFIED ;
external_property_gender_suitability = UNVERIFIED
equipment_default_quantity = 1
shuttle_capacity_seats = 12 ; shuttle_vehicle_count = 3 ; shuttle_travel_min
= 25 ; transport_hours = 06:00-22:00   (PLACEHOLDERS, not facts)
accom_plan_c_weights = cohesion 0.5, cost 0.3, travel 0.2 (placeholder)
include_leaders_escorts_in_demand = true
count_unknown_accommodation_as_possible = true (reported separately from
confirmed)
late_arrival_policy = PENDING_DIRECTOR_DECISION

11. GIT AND INTEGRATION
- Branches: main (demo-ready, protected), develop (integration), pN/NN-slug
feature branches. PRs go to develop only, rebased, squash-merged, reviewed by
another person (cross-owner changes need the owner). develop merges to main
only at checkpoints (end of each round, code freeze).
- backend/requirements.txt has one section per owner (# --- P1 ---, # --- P2
---, # --- P3 ---); append only inside your own section. Frontend
dependencies: PR labelled `deps`; everyone runs `npm ci` after pulling.
Commit lockfiles.
- Env: .env.example is the single source; add every new variable there with a
comment. Never commit secrets. Ports: API 8000, web 5173. DB file
backend/festos.db is gitignored.
- No Alembic. A schema change means editing the model and running `make
reset`. After round 1 only additive columns, announced in team chat. Append
to docs/INTEGRATION_LOG.md after each merge: date, prompt, endpoints,
public.py functions, breaking notes.
- CI must pass: ruff, pytest, tsc --noEmit, vitest, generated types up to
date.

12. DEFINITION OF DONE (PR checklist for every prompt)
[ ] docs/api/<module>.md contract pushed first  [ ] router + service + engine
+ models + schemas, mutations through ChangeService, audit and Why cards
where relevant  [ ] public.py with summary/risks/dependents  [ ] seeder or
fixtures  [ ] pytest including the listed edge cases  [ ] lint, typecheck and
type generation green  [ ] INTEGRATION_LOG entry  [ ] no hardcoded demo
numbers, no secrets, sensitive fields masked

13. SPEC DECISIONS ALREADY MADE (do not reopen)
- Never force 2,400. Show raw 2,836, declared 2,842 and narrative 2,400 with
a reconciliation that explains or flags the remainder.
- One cleaning buffer assumption (default 180 min), shown in the UI.
- Confirmed and possible accommodation demand are always reported separately.
```

```
- Three accommodation plans A, B and C (C is new and must be implemented).
- Hard accessibility, safety and privacy constraints can be overridden only
by the Festival Director, with a reason, and always appear in the override
register.
- Late-arrival wave and wildcard finals need Director approval.
- Duplicates are never auto-merged; merges are reversible.
- The pack has no event times, venues, judges or team IDs, so any generated
schedule is labelled ASSUMED and judge slots show as gaps.
- Vendor capacity is evaluated per meal slot, venue and dietary category, not
only in aggregate.
- Zero venue overlaps does not mean zero conflicts: all ten conflict types
are checked.
```

B. Roadmap at a glance

#	Owner	Section	Depends on	Branch
1	P1	Foundation: repo, schema, ChangeService, registries, frontend scaffold	Step 0	p1/01-foundation
2	P2	Data import, validation, quarantine, assumptions register, seed	1	p2/02-import
3	P3	Auth, RBAC, masking, audit, approvals, break-glass, design system, shell	1	p3/03-security-ui
4	P1	Participants, contingents, unknown-state, duplicate workbench (backend)	1, 2 (soft)	p1/04-participants
5	P2	Accommodation engine: constraints, plans A/B/C, shortages, Why-not	1, 2	p2/05-accom-engine
6	P3	UI: import, data quality, assumptions, participants, contingents, reconciliation, unknowns	2, 3, 4	p3/06-ui-data
7	P1	Events and venues, scheduling engine, conflict detector, resolutions	1, 2	p1/07-scheduling

#	Owner	Section	Depends on	Branch
8	P2	Accommodation operations: live plan, overrides, check-in/out, repair	3, 5	p2/08-accom-ops
9	P3	UI: accommodation and check-in desk	5, 8	p3/09-ui-accom
10	P1	Inventory ledger, equipment, merchandise QR and duplicate-claim defence	1, 4	p1/10-inventory
11	P2	Vendors, food coverage, shutdown response, notifications, finance	1, 2, 3	p2/11-vendors
12	P3	UI: schedule, logistics, merchandise counter, vendors, finance	7, 10, 11	p3/12-ui-ops
13	P1	Staff and shifts, duty-of-care and shuttle planner	4, 7, 8	p1/13-staff-transport
14	P2	Change impact engine, incident and disruption console, replanner	4, 7, 8, 10, 11, 13	p2/14-impact
15	P3	UI: disruption console, impact tree, staff, transport, notifications	13, 14	p3/15-ui-incidents
16	P1	Dashboard, attention queue, risk heatmap, decision log, scenario simulator	14	p1/16-dashboard-sim
17	P2	Unknowns burn-down, fest pre-mortem, forecasting	5, 14, 16	p2/17-foresight
18	P3	UI: overview, simulator, pre-mortem, burn-down, decision log	16, 17	p3/18-ui-insight
19	P1	AI command center and all reports/exports	14, 16	p1/19-commands-reports
20	P2	Hardening: full test matrix, concurrency, security review	1 to 19	p2/20-hardening
21	P3	PWA offline check-in/scan, volunteer view, command bar and report UI, polish	8, 10, 19	p3/21-pwa-polish

#	Owner	Section	Depends on	Branch
22	P1	Final integration, twist playbook, Docker deployment, code freeze	1 to 21	p1/22-integration
23	P2	Submission pack: README, setup, credentials, AI disclosure, sample plans	20, 22	p2/23-submission
24	P3	E2E tests of the seven live scenarios, demo script, pitch, rehearsal	21, 22	p3/24-qa-pitch

Suggested clock (24 h): round 1 (prompts 1 to 3) hours 0 to 2.5; round 2 (4 to 6) to hour 6; round 3 (7 to 9) to hour 10; round 4 (10 to 12) to hour 14, absorbing the hour-12 twist with the Twist Prompt in Section E; round 5 (13 to 15) to hour 17.5; round 6 (16 to 18) to hour 20; round 7 (19 to 21) to hour 22 (code freeze); round 8 (22 to 24) is integration, docs and rehearsal only, with no new features. If time runs short, cut from the bottom of this priority order: import and reconciliation, accommodation with explainability, scheduling and conflicts, disruption console and impact engine, merchandise ledger, RBAC and audit, vendors, then the advanced innovations.

C. The 24 prompts

Each prompt block is self-contained: paste it into Antigravity as written. The line above each block says who runs it, what it depends on, and which files that person owns, so file collisions are impossible when everybody follows the ownership map.

Prompt 1 · Person 1 · Foundation: repo, schema, ChangeService, registries, frontend scaffold

Branch `p1/01-foundation` · **Depends on** Step 0 · **Unblocks** everything · **Creates** repo skeleton, `core/*` (P1 parts), `schemas/common.py`, all `models/*`, frontend scaffold

```
READ docs/FESTOS_CONTEXT.md and docs/spec/ (problem statement + master
specification) fully. Branch: p1/01-foundation.
Target: push an INTERFACE-FIRST commit (core/, schemas/common.py, models/,
registries) within 30 minutes because P2 and P3 are waiting on it, then
finish the rest.

GOAL: a runnable, tested skeleton on which three people build in parallel
```

```
without touching each other's files.

TASKS
1. Repo and tooling: monorepo layout from context section 3 (create every
module folder with a .gitkeep and an OWNER header), .gitignore, .env.example
(DATABASE_URL, JWT_SECRET, JWT_EXPIRE_MIN, AUTH_DISABLED, FESTOS_TZ,
FESTOS_NOW, DATA_DIR, SEED_ON_START, LLM_PROVIDER, LLM_API_KEY, ACCOM_SOLVER,
SCHED_SOLVER, QR_SECRET, VITE_API_URL, CORS_ORIGINS), CODEOWNERS from the
ownership map, .github/PULL_REQUEST_TEMPLATE.md (Definition of Done
checklist), .github/workflows/ci.yml (ruff, pytest, tsc --noEmit, vitest,
gen-types drift check), Makefile targets: setup dev dev-api dev-web test lint
gen-types seed reset demo clean. docker-compose.yml with backend and frontend
Dockerfiles (dev profile). pre-commit config.
2. backend/app/core: config.py (pydantic-settings); db.py (engine factory,
SQLite WAL + foreign_keys + busy_timeout pragmas, session dependency,
`use_scenario(scenario_id)` contextvar that redirects sessions to
scenarios/<id>.db); errors.py (exceptions + handlers producing the error
envelope); clock.py (now(), FESTOS_NOW, set_demo_time); scoring.py (attention
score, urgency buckets, heatmap bucket helper); bus.py; registry.py (change
handlers, seeders, participant sections, iter_public_modules() discovery);
idempotency.py (dependency storing request hash + response in
idempotency_key; replays return the stored response); change_service.py
implementing context section 7 exactly (savepoint transaction,
expected_version check, dry_run, PENDING_APPROVAL path calling a stubbed
approvals interface, impact-hook registry defaulting to no-op, writes
change_set + audit through core.audit.record()). Create minimal stubs with
header `OWNER: P3` for audit.py (append-only writer), security.py, rbac.py,
masking.py, deps.py; get_actor returns a dev FESTIVAL_DIRECTOR while
AUTH_DISABLED=true.
3. schemas/common.py exactly as context section 8 (Pydantic v2) plus Envelope
and Page helpers.
4. models/: every canonical and extra table from context section 6, as
SQLModel classes in the owner files from the ownership map. Include FKs,
indexes, version/created_at/updated_at, Tri and UNKNOWN defaults,
source_row_id, JSON columns for explanation/snapshot payloads, and these
guarantees: unique(canonical_participant_id, item_id) on merch_claim;
stay_assignment has plan_id, bed_key, start_at, end_at with unique(plan_id,
bed_key, start_at). Cover fields the spec needs: participant (gender, diet,
housing_preference, accessibility_needs json, accommodation_requested Tri,
arrival_at, departure_at, dob/age, role
PARTICIPANT|LEADER|ESCORT|JUDGE|VOLUNTEER|STAFF, status,
canonical_participant_id), room (block, floor, bed_count, accessible,
gender_class, status, cleaning_ready_at), external_property
```

```
(cost_per_bed_night, distance_km, transport_limited, accessibility_verified,
gender_suitability_verified), inventory_item (kind equipment|merch, size,
condition), vendor (dietary_declared json, menu json, hygiene_valid_from/to,
status), staff (skills json), event (solo_team, rounds, duration_min,
requirements json). Keep them minimal but complete; later prompts add columns
additively in their own files.
5. app/main.py: auto-discover api/v1/*.py routers (any module exporting
`router`), mount under /api/v1, request-id middleware, CORS, error handlers,
/health and /api/v1/meta (version, now(), module list, auth_disabled). Add
api/v1/_example.py demonstrating router -> service -> engine -> ChangeService
with a test.
6. seed/registry.py and `python -m app.seed {reset,load}` (drop/create
tables, run seeders by order).
7. Tests: ChangeService (success, rollback leaves no partial state, version
conflict, dry_run writes nothing, pending approval), idempotency replay,
scoring buckets, scenario DB redirect, model constraints (duplicate merch
claim rejected, overlapping stay rejected).
8. Frontend scaffold: Vite + React + TS strict + Tailwind + Router + TanStack
Query; src/api/client.ts (envelope unwrap, JWT header, Idempotency-Key
helper, typed ApiError); src/app/registry.ts that auto-registers features
through import.meta.glob('../features/*/index.ts') where each feature exports
{id, label, group, path, icon, permissions, routes}; nav groups follow spec
section 18 (Overview, Participants, Contingents, Check-in, Accommodation,
Events & Venues, Logistics, Merchandise, Vendors & Food, Staff & Shifts,
Transportation, Incident Center, Impact Analysis, Scenario Simulator, Fest
Pre-Mortem, Unknowns Burn-down, AI Command Center, Data Reconciliation,
Reports, Audit & Permissions); placeholder AppShell; MSW and vitest
configured; `make gen-types` using openapi-typescript against the running
API.
9. Docs: docs/DECISIONS.md (ADR template), docs/INTEGRATION_LOG.md,
docs/api/_template.md, backend/README.md, root README skeleton.

ACCEPTANCE
- `make setup && make dev` starts the API on 8000 and the web app on 5173;
GET /health works; OpenAPI served at /docs.
- `make lint test gen-types` pass locally and in CI.
- Adding backend/app/api/v1/foo.py with a router mounts it with ZERO edits to
main.py; adding frontend/src/features/foo/index.ts appears in the nav with
ZERO edits to shared files.

GIT: logical commits, push p1/01-foundation early (interface-first), PR into
```

```
develop, tag v0.1-foundation, then message the team with the exact pull-and-
run commands.
DO NOT implement business features, import data, or write module code owned
by others.
```

Prompt 2 · Person 2 · Data import, validation, quarantine, assumptions register

Branch `p2/02-import` · **Depends on** 1 · **Unblocks** 4, 5, 6, 7 · **Owns** modules `imports`, `assumptions`, `models/ops.py`, `engines/importer/`, `data/mapping/`

```
READ docs/FESTOS_CONTEXT.md fully. Branch: p2/02-import. Follow ownership,
ChangeService, public.py and contract-first rules (push docs/api/imports.md
and docs/api/assumptions.md first).

GOAL: turn the messy data pack into trustworthy canonical data, without
hiding or deleting anything uncertain.

TASKS
1. PROFILE FIRST. Read every CSV in data/raw (participants, registrations,
contingents, events, venues, core_team, hostels, external_stays,
inventory_logistics, inventory_merch, vendors, README.txt). Write
docs/DATA_PROFILE.md: columns, types, null counts, label variants, date/time
formats, anomalies, cross-file key relationships. Do not assume headers;
derive them.
2. engines/importer: per-file column-mapping config in
data/mapping/<file>.yaml (raw header or synonyms -> canonical field),
tolerant parsing for mixed date/time formats and timezone normalisation (to
UTC, display IST), label normalisers (gender, diet, accessibility,
accommodation_requested -> Tri, housing preference). Blank or unrecognised
values become the literal UNKNOWN, never NO. Raw rows are stored untouched in
raw_record (immutable) before anything else.
3. Pipeline: upload -> import_batch -> staging -> validation ->
quarantine_row for invalid rows (with reason code) -> human review -> promote
to canonical tables through ChangeService handler `imports.promote`
(audited). Re-import is idempotent via batch content hash. Provide dry-run
and batch history. Clear errors for missing columns and malformed CSV.
4. Validation rules (flag, never silently drop): arrival not before departure
(spec says 3 invalid intervals; verify), declared contingent strength vs
actual participant count (compute per contingent; spec reference: declared
total 2,842 vs 2,836 records), leader/escort present in the participant file
(spec reference: only 3 of 38 each; verify), age below minor threshold,
registrations pointing at unknown participant/event, duplicate-looking rows
(detection itself is Prompt 4), vendor menu vs declared dietary label
mismatch (e.g. a Jain thali menu with no Jain declared), hygiene certificate
dates vs festival dates, hostel and external bed totals vs spec (710 / 740),
events lacking times or venues, team registrations lacking team IDs, staff
appearing as participants, equipment condition inconsistencies.
5. Data-quality service: issues per file/field/severity, counts of UNKNOWN
per field with potential operational impact text, exportable
data_quality_report; reconciliation summary shell (raw count, narrative
expected count from assumption, declared total, invalid, quarantined) that
Prompt 4 extends.
6. Assumptions register: assumption table CRUD through ChangeService
```

```
(history, owner, source, reason, last updated, impact-if-changed, validation
of typed values); helper get_assumption(key) with cache invalidation on
change; seed ALL keys in context section 10 with defaults; endpoint that
previews which modules read a key.
7. Endpoints: POST /imports (multipart), GET /imports, GET /imports/{id}, GET
/imports/{id}/issues, POST /imports/{id}/promote, GET /data-quality, GET/PUT
/assumptions, GET /assumptions/{key}/history.
8. Seeders (order 10) so `make reset` loads the full data pack end to end.
Generated stable IDs per context section 4.
9. public.py for imports and assumptions (risks: unresolved quarantine,
UNKNOWN-heavy critical fields).

TESTS: malformed CSV, missing columns, quarantine rows carry reasons, UNKNOWN
stays UNKNOWN, idempotent re-import, raw records preserved byte-for-byte,
golden counts compared with spec reference numbers (assert plausibility and
print differences; do not hardcode).
ACCEPTANCE: `make reset` produces a populated database; /data-quality
explains the 2,836 vs 2,400 vs 2,842 situation as far as imports can; every
assumption editable and audited.
DO NOT implement duplicate detection or any engine owned by others.
```

Prompt 3 · Person 3 · Security, audit, approvals, design system, app shell

Branch `p3/03-security-ui` · **Depends on** 1 · **Unblocks** 6, 8, 9 and every UI · **Owns** `core/{audit,security,rbac,masking,deps}`, `models/audit.py`, modules `auth`, `audit`, `approvals`, `overrides`, `seed/users.py`, all of `frontend/` from now on

```
READ docs/FESTOS_CONTEXT.md fully. Branch: p3/03-security-ui. Replace the P3-
owned stubs created in Prompt 1. Push docs/api/auth.md, audit.md,
approvals.md, overrides.md first.

BACKEND
1. Auth: JWT with role claims, argon2/bcrypt hashing, login, refresh, me,
basic lockout, `AUTH_DISABLED` honoured only for tests. Replace
get_actor/deps so every router can use require_permission('<module>:
<action>').
2. RBAC: core/rbac.py holds the full matrix for the nine roles (Festival
Director full overview and high-risk approvals; Accommodation Lead hostels,
properties, accessibility, room allocation; Event Lead events, schedules,
venues, judges, volunteers; Logistics Manager equipment and inventory;
Merchandise Manager entitlement and issue; Food Ops Manager vendors, menus,
food coverage; Finance Manager costs, contracts, settlements; Volunteer
limited check-in and merchandise scanning; Auditor read-only audit).
Permission strings per context section 9. GET /auth/permissions for the
frontend.
3. Field-level masking (core/masking.py): serializer helper
mask_fields(model, actor) for phone, email, medical, accessibility details,
room data, guardian data; consistent partial masking; export restrictions
(export_allowed decorator); masking also applied to logs and CSV exports.
4. Break-glass: POST /auth/break-glass {entity, field, reason, minutes};
mandatory reason, short TTL, scoped to one entity/field, audited;
break_glass_grant table; automatic expiry; list active grants.
5. Audit hardening: audit_event append-only (reject UPDATE/DELETE at app and
DB trigger level), hash chain (prev_hash) with a verify endpoint, fields per
spec 5.11 (action, previous state, new state, user, role, timestamp, reason,
approval, related incident/change id), query API (filters, entity timeline,
CSV export respecting masking).
6. Approvals framework: approval table, policy config (which risk kinds need
which approver roles, mandatory reason), endpoints list/approve/reject,
integration with ChangeService PENDING_APPROVAL (approving executes the
stored change). Override register: override table, core.overrides.record(...)
helper (used by P1/P2 modules), GET /overrides with filters.
7. seed/users.py (order 20): one seeded user per role plus extra volunteers;
passwords and emails documented in docs/TEST_CREDENTIALS.md.

FRONTEND DESIGN SYSTEM AND SHELL (you now own frontend/)
8. Tokens and dark ops-centre theme with a light option; severity colours
must also have icons/labels (never colour-only). Components in
src/components/ui: Button, Card, Badge (severity, status, Tri state with
distinct icons for YES/NO/UNKNOWN), DataTable (server paging, sort, filter,
```

```
virtualised), Drawer, Modal, ConfirmWithReason (mandatory reason, used for
every override), Toast, Tabs, EmptyState, Skeleton, KpiTile, SeverityPill,
MaskedField (click opens break-glass dialog), WhyCard (renders DecisionCard),
ErrorBoundary.
9. Layout: AppShell with sidebar built from the feature registry grouped per
spec section 18 and filtered by permissions, top bar with demo clock, user
menu, approvals bell, global command-bar slot, keyboard shortcuts (/ search,
g then o overview). Login page and route guards.
10. Feature `audit` (Audit & Permissions page): audit timeline with filters,
override register, approvals inbox, break-glass log, role-permission matrix
viewer, chain-verify button.

TESTS: pytest parametrised allow/deny matrix for every role x sensitive
endpoint, masking per role, break-glass expiry, audit immutability (UPDATE
fails), hash-chain verification, approval executes stored change; vitest for
ConfirmWithReason, MaskedField, Badge, route guards.
ACCEPTANCE: log in as each seeded role and see a different nav and masked
data; an override without a reason is impossible; audit rows cannot be
altered.
DO NOT build module screens beyond the audit page.
```

Prompt 4 · Person 1 · Participants, contingents, UNKNOWN state, duplicate reconciliation workbench

Branch `p1/04-participants` · **Depends on** 1; soft on 2 (use factories until import lands) · **Owns** modules `participants`, `contingents`, `reconciliation`, `core/events` subscribers for participants

```
READ docs/FESTOS_CONTEXT.md fully. Branch: p1/04-participants. Push
docs/api/participants.md, contingents.md, reconciliation.md, unknowns.md
(field-verification parts) first.

GOAL: one canonical participant record that every module references, plus an
honest reconciliation of the 2,836 / 2,400 / 2,842 mismatch.

TASKS
1. Participants: list/search/filter (fuzzy name, contingent, status, event,
diet, accessibility, UNKNOWN filters, role, arrival window), profile CRUD
through ChangeService, status machine (EXPECTED, ARRIVED, CHECKED_IN,
CHECKED_OUT, CANCELLED, NO_SHOW, WITHDRAWN, INVALID, MERGED) that publishes
participant.status_changed on the bus so other modules react without being
edited. History timeline from audit.
2. Participant 360: GET /participants/{id}/360 aggregates contingent,
registrations/events, diet, accessibility, merch entitlement, operational
status and any sections registered through register_participant_section
(accommodation, merchandise, transport register themselves later). Sensitive
sections are masked by role.
3. Contingents (38): profile, declared vs actual strength, leader and escort
details with FOUND/MISSING in participant data, arrival/departure
coordination, member attendance, event participation summary, dietary
summary, merchandise status, group alerts, fragmentation analysis and leader
proximity (read stay assignments through accommodation public.py when
available, otherwise return empty with a warning). GET
/contingents/{id}/status gives the full operational view.
4. UNKNOWN as first-class: field_verification service (list the unknown queue
by field, owner, impact text; record verified values with source, verifier,
timestamp through ChangeService; data-owner assignment; confirmation
history). Signed, expiring, field-scoped confirmation tokens for contingent
leaders and desk volunteers: POST /unknowns/confirm/{token}. Ranking in this
prompt is a simple rule (count x criticality); value-of-information ranking
arrives in Prompt 17.
5. Duplicate engine (engines/dedupe, pure): blocking by contingent and
phonetic name to avoid O(n^2); RapidFuzz name similarity combined with
contact, age/dob, contingent and overlapping registrations; confidence score
with explainable signals; classes EXACT, LIKELY, POSSIBLE. Names repeat
heavily in this pack and contacts may be unique, so name similarity alone
must NEVER reach LIKELY. Spec reference: ~7 likely and ~305 possible pairs
(verify, do not hardcode). Persist duplicate_candidate rows.
6. Workbench API: counts panel (raw, narrative expected, declared, exact,
likely, possible, withdrawn, invalid, unresolved, quarantined) with a
reconciliation waterfall that explains the gap or shows the unexplained
```

```
remainder explicitly; side-by-side compare with linked registrations and
claims; merge (canonical id kept, losers MERGED, registrations and claims re-
pointed, full pre-state saved in merge_record), keep-separate decision, undo
merge restoring the exact prior state, merge history. Merge is blocked for
review if both records already hold conflicting merch claims. Edge cases:
same name different people, leader who is also a competitor, escort who is
also a judge, staff counted as participants (flag them).
7. Register a participants public.py (summary, risks: unresolved duplicates,
unknown-heavy arrivals, late/missing arrivals; dependents: participant ->
contingent, registrations, stay, claims).

TESTS: undo merge restores byte-identical rows; same-name-different-contact
never auto-merges; reconciliation numbers add up; status change publishes bus
event once; masked fields hidden for Volunteer role.
ACCEPTANCE: after `make reset`, an organiser can open any participant 360,
see UNKNOWN fields as UNKNOWN, merge a likely duplicate with a reason, and
undo it.
DO NOT implement accommodation or merchandise logic here.
```

Prompt 5 · Person 2 · Accommodation engine: constraints, plans A/B/C, shortages, Why-not

Branch `p2/05-accom-engine` · **Depends on** 1, 2 (4 soft) · **Unblocks** 8, 9, 14, 17 · **Owns** `engines/accommodation/`, module `accommodation` (planning half), `models/accommodation.py`

```
READ docs/FESTOS_CONTEXT.md and spec sections 3.3-3.5, 4.2, 6.3, 12.1 fully.
Branch: p2/05-accom-engine. Push docs/api/accommodation.md first.

GOAL: an explainable, deterministic accommodation planner that never hides a
violated constraint. Engine code in engines/accommodation is pure (dataclass
snapshots in, plan objects out); services/accommodation wraps it with DB
access.

TASKS
1. Demand: build per-participant stay intervals from arrival/departure;
report CONFIRMED demand (accommodation_requested = YES) and POSSIBLE demand
(UNKNOWN) separately; invalid intervals are excluded AND listed, never
dropped silently; leaders/escorts included per assumption. Bed-demand
timeline and peak (spec reference ~1,601 at 30 Oct 6 PM; verify) via
bed_demand_timeline.
2. Rolling occupancy: bed timeline with check-in/check-out windows and
cleaning buffer from get_assumption('cleaning_buffer_minutes'); room
status/readiness/maintenance respected; no double-booking.
3. Stage A hard filters: wheelchair need -> accessible rooms only; ground-
floor need -> ground floor only; near-washroom and visual-assistance needs ->
NEEDS_MANUAL_VERIFICATION (never auto-satisfied); privacy/single-room needs
honoured; declared gender housing honoured; non-binary, prefer-not-to-say,
blank or UNKNOWN participants are NEVER put in a gendered block automatically
(eligible only for single/neutral/private options, otherwise unplaced with a
reason); external properties with unverified accessibility or gender
suitability are ineligible for participants who need them; minors only placed
externally when guardian approval, escort and transport conditions are
modelled, and never in a limited-transport property; early/late events and
transport windows per rule 6.
4. Stage B contingent packing: largest contingents first, same block/floor
where possible, leader near the group, escorts housed separately but nearby,
minimise splits and record the reason for each split.
5. Stage C external overflow: cost per bed-night from the property, travel
time, transport window, safety; budget from the external_budget_inr
assumption. When the budget is unset run in budget-unset mode: show costs,
flag every external placement, skip over-budget rejection.
6. Stage D shortage analysis: every unplaced participant gets a blocking-
reason set from a fixed taxonomy (HARD_ACCESSIBILITY,
GENDER_OR_HOUSING_UNKNOWN, TRANSPORT_RESTRICTED, NO_ELIGIBLE_PROPERTY,
BUDGET, MISSING_INFO, CAPACITY) with rejected rooms/properties and a
counterfactual: what would change if the unknown field were confirmed (re-run
eligibility with each possible value).
7. Stage E three plans: A cohesion-first, B cost-first, C balanced (weights
```

```
from accom_plan_c_weights). Greedy + local-repair pass (swap/move to cut
splits without breaking hard rules). Optional OR-Tools CP-SAT behind
ACCOM_SOLVER with a time limit and greedy fallback. Never describe results as
optimal. Deterministic: stable ordering and a plan hash.
8. Explainability: each assignment stores constraints satisfied, soft
constraints relaxed + reason, rejected alternatives (top k), cost,
verification status, approval info as JSON on stay_assignment.explanation.
Plan metrics: on-campus, external, unplaced, splits, cost, safety flags,
accessibility issues, relaxed constraints, minors external count.
9. API: POST /accommodation/plans/generate {kinds, params}, GET
/accommodation/plans, GET /accommodation/plans/{id}, GET
/accommodation/plans/compare?ids=, GET /accommodation/plans/{id}/shortages,
GET /accommodation/why-not/{participant_id}?plan_id=, GET
/accommodation/demand-timeline, GET rooms and properties (management CRUD
arrives in Prompt 8). Candidate plans are NOT live.
10. Report functions for accommodation_plan_A/B/C.csv, shortage_report.csv,
accommodation_summary.csv, bed_demand_timeline.csv (consumed by Prompt 19).
Decision cards for each plan. Seeder (order 30) for rooms and properties.

TESTS (pytest + hypothesis): no capacity exceeded; no bed double-booked; hard
constraints never violated across random inputs; UNKNOWN never silently
assigned; cleaning interval respected; every unplaced participant has
documented reasons; hand-calculated small fixtures; golden run on the data
pack printing metrics next to the prototype references (861 on campus, 920
external, 360 unplaced for cohesion-first) as sanity context, not as targets.
ACCEPTANCE: generating A, B and C returns three comparable plans with
violated/relaxed constraints visible and a working Why-not for any unplaced
participant.
DO NOT touch live-plan commit, check-in or overrides (Prompt 8).
```

Prompt 6 · Person 3 · UI: import, data quality, assumptions, participants, contingents, reconciliation, unknowns

Branch `p3/06-ui-data` · **Depends on** 2, 3, 4 (use MSW mocks from `docs/api` until they merge) · **Owns** `frontend/src/features/{imports,assumptions,participants,contingents,reconciliation,unknowns}`

```
READ docs/FESTOS_CONTEXT.md fully. Branch: p3/06-ui-data. Use generated types
(make gen-types) and MSW handlers built from docs/api/*.md until the real
endpoints are merged; switch to the real API when available and keep contract
tests.

BUILD (each as a feature folder with index.ts registering nav, routes and
permissions)
1. Import and data quality (Data Reconciliation group): upload wizard (files
-> detected mapping -> validation summary -> quarantine review -> promote
with reason), batch history, issues table filterable by file/field/severity,
UNKNOWN heat per field with impact text, download data_quality_report.
2. Assumptions register page: table of every assumption (name, current value,
source, owner, last updated, reason, impact-if-changed), inline edit using
ConfirmWithReason, history drawer, banner whenever a placeholder/unverified
assumption is in use; surface the three open decisions (external budget,
late-arrival policy, shuttle assumptions).
3. Participants: virtualised searchable list with filters (contingent,
status, event, diet, accessibility, UNKNOWN, role), saved filters, bulk
selection, 360 drawer with tabs (profile, events, accommodation, merchandise,
transport, history) where Tri-state and UNKNOWN are visually distinct and
sensitive fields use MaskedField.
4. Contingents: grid of 38 with declared vs actual strength delta, missing
leader/escort flags, alerts; detail page with arrivals, members, events, diet
summary, merch status, accommodation distribution and split reason.
5. Data Reconciliation workbench: counts panel and waterfall (raw vs
narrative vs declared, with the unexplained remainder shown, not hidden),
duplicate queue sorted by confidence with signal chips, side-by-side compare
with linked registrations/claims, Merge / Keep separate / Undo using
ConfirmWithReason, merge history.
6. Unknowns queue: grouped by field and owner, inline verification form with
source + timestamp, leader confirmation-link generator, confirmation history.
(Value-of-information ranking is added in Prompt 18.)

UX RULES: every list is keyboard navigable, loading skeletons, empty states
with next action, errors show the API error code, no raw JSON.
TESTS: vitest per page against MSW; one Playwright smoke per feature (login
as Director, load page, perform the primary action).
ACCEPTANCE: a Director can import the pack, review quarantine, edit an
assumption with a reason, merge and undo a duplicate, and verify an UNKNOWN
field, all from the UI.
DO NOT edit backend files; if a contract is missing, open an issue and tag
the owner.
```

Prompt 7 · Person 1 · Events, venues, scheduling engine, conflict detector, resolutions

Branch `p1/07-scheduling` · **Depends on** 1, 2 · **Unblocks** 12, 13, 14 · **Owns** modules `events`, `venues`, `schedule`, `engines/scheduling/`, `models/events.py`

READ docs/FESTOS_CONTEXT.md and spec sections 3.8-3.9, 4.3, 5 (Feature 4), 12.2 fully. Branch: p1/07-scheduling. Push docs/api/events.md, venues.md, schedule.md first.

GOAL: a scheduler and conflict detector that is honest about missing data (no event times, no venue assignments, no judges, 1,810 team registrations without team IDs) and never silently drops anyone.

TASKS
1. Events: profile (duration, type, solo/team, rounds, requirements for judges, volunteers, equipment), event_requirement, registrations view, cancel/merge/extend-duration actions through ChangeService. Venues: capacity (max simultaneous competitors per heat), availability windows, operating hours, accessibility, equipment compatibility, setup and teardown time, editable feature tags (tags are missing in the pack). travel_time matrix defaulting to venue_travel_min with per-pair overrides. Venue-event compatibility is an editable ASSUMED mapping.
2. Judges: no judge data exists. Provide a judge registry with manual entry and generate explicit UNASSIGNED judge slots shown as gaps. Never invent judges. Flag a judge from a participating college or who is also a competitor.
3. Team reconciliation: where team events lack team IDs, derive provisional teams (contingent + event) labelled ASSUMED and editable; report how many registrations were affected.
4. Scheduler (engines/scheduling, pure): generate sessions for preliminary, semifinal, final rounds using semifinal_ratio, finalist_ratio/min, round_result_buffer_min, setup/teardown buffers, check-in buffer and travel buffers. Heats are bounded by venue capacity. Use a participant-conflict graph to cluster heats and minimise clashes; optional OR-Tools CP-SAT with NoOverlap intervals per venue and per participant behind SCHED_SOLVER with a time limit and a greedy fallback. Support arrival waves (early, main, late-arrival wave). A late-arrival wave and wildcard finals are fairness decisions: create them as approval-required proposals, not automatic facts. Every generated schedule is labelled ASSUMED.
5. Unscheduled reporting: the spec's prototype left 987 registrations unscheduled and 28 later-round sessions unplaced. Your engine must report scheduled vs unscheduled registrations, each unscheduled entrant with a reason and alternatives, and the comparison against the prototype numbers. Never discard participants.
6. Conflict detector (sweep-line, overlap test start_a < end_b and start_b < end_a expanded by setup/teardown/travel). All ten types: participant clash, team clash, judge double-booking, volunteer double-booking, venue double-booking, capacity violation, equipment conflict, insufficient travel time,

```
setup/teardown conflict, operating-hour conflict. Severity classification
(blocking vs warning), persisted in the conflict table with OPEN, RESOLVED,
ACCEPTED. Equipment availability is read through an EquipmentAvailabilityPort
(default adapter reads inventory tables; Prompt 10 refines it).
7. Resolution recommender: candidate venue/time moves ranked by conflicts
resolved, new conflicts created, people affected; apply through handlers
schedule.move_session, schedule.change_venue, event.cancel, event.merge
(blocking conflicts need approval). schedule_version snapshots with diff and
restore. Round dependencies and delayed results cascade correctly.
8. API: events and venues CRUD, POST /schedule/generate, GET
/schedule/versions, GET /schedule/sessions, GET /schedule/conflicts, GET
/schedule/conflicts/{id}/resolutions, POST apply resolution, GET
/schedule/unscheduled, GET /schedule/timeline?from&to (Gantt data by venue).
Sessions expose volunteers_required and equipment_required for Prompt 13.
9. public.py: summary, risks (blocking conflicts in the next four hours,
unscheduled entrants, judge gaps), dependents (event -> venue, equipment,
staff, participants). Report functions for schedule_plan.csv,
conflicts_report.csv, unscheduled_entrants.csv, resource_load_timeline.csv.
Seeder for events and venues.

TESTS: interval maths unit tests; property test that generated schedules have
no venue/participant overlaps given the inputs; every conflict type has a
fixture; venue capacity reduced, duration increased and delayed round results
each produce correct cascades; zero venue overlaps is never reported as zero
conflicts.
ACCEPTANCE: generate a schedule from the pack, see conflicts by type, apply a
ranked resolution, and restore a previous schedule version.
DO NOT implement staffing or equipment ledgers.
```

Prompt 8 · Person 2 · Accommodation operations: live plan, overrides, check-in/out, repair

Branch `p2/08-accom-ops` · **Depends on** 3, 5 · **Unblocks** 9, 13, 14, 21 · **Owns** modules `accommodation` (operations half), `checkin`, `engines/repair/`

```
READ docs/FESTOS_CONTEXT.md and spec sections 3.1, 3.5, 8 (accommodation edge cases), 6.6 fully. Branch: p2/08-accom-ops. Extend docs/api/accommodation.md and push docs/api/checkin.md first.

GOAL: turn candidate plans into a live, auditable operation that survives cancellations, late arrivals, closures and two people clicking at once.

TASKS
1. Resource management through ChangeService: rooms and external properties (status READY, OCCUPIED, CLEANING, MAINTENANCE, CLOSED; floor/block; accessible; gender class; pricing; transport windows; safety info; verified flags). Cleaning completion marks a room READY; checkout starts the cleaning timer from cleaning_buffer_minutes.
2. Live plan: commit a candidate plan to LIVE through approvals (Accommodation Lead proposes; Director approves when external cost exceeds the budget or any hard-constraint relaxation exists). Freeze logic plus allocate_incremental() for contingents or participants arriving after freeze that preserves every approved assignment (minimal change). Manual assign/move returns Violation[] and requires a reason; overriding a hard accessibility, safety or privacy constraint is Director-only, needs a reason and always writes to the override register via core.overrides.record. Decision card for every assignment or override.
3. Concurrency: two organisers allocating the same room or bed must yield one success and one clear 409 (expected_version plus the unique bed/interval guard).
4. Arrivals and check-in/out: arrival tracking, check-in by id/QR/search (verifies the stay, blocks if the room is not ready and offers alternatives), check-out, early/late arrival updates. Cancellation and no-show subscribe to participant.status_changed, release beds and SUGGEST (never auto-apply) reallocations for the unplaced. Idempotent check-in with client_op_id, plus POST /checkin/sync accepting a batch of offline operations and returning per-op APPLIED | DUPLICATE_IGNORED | CONFLICT with explanations (server state wins, conflict explained).
5. Minimal-change repair engine (engines/repair): replan_accommodation(session, plan_id, closed_room_ids | closed_block | closed_property, strategy) finds affected participants, re-places them with the fewest relocations while preserving cohesion and hard constraints, and returns a candidate plan plus a diff (people moved, splits added, cost delta). Prompt 14 uses it for hostel closure and Prompt 16 for the simulator.
6. Register participant 360 section 'accommodation' and a public.py (summary: occupancy, available beds, unplaced; risks: unplaced participants, accessibility needs unconfirmed with arrival within six hours, capacity shortfalls, minors in external stays without guardian approval; dependents:
```

```
room -> participants, property -> participants, participant -> room).

TESTS: concurrent allocation of the same bed yields exactly one success (use
threads); checkout then immediate re-allocation is blocked until cleaning
completes; cancellation frees the bed and creates a suggestion, not a change;
hard override without Director role or reason is rejected; repair after
closing a block never violates hard constraints and reports a correct diff;
offline sync is idempotent when replayed.
ACCEPTANCE: commit plan C to LIVE with approval, close a block, receive a
minimal-change repair proposal, approve it, and see the live assignments
change with audit entries.
DO NOT write frontend code.
```

Prompt 9 · Person 3 · UI: accommodation and check-in desk

Branch `p3/09-ui-accom` · **Depends on** 5, 8 · **Owns** `frontend/src/features/{accommodation,checkin}`

```
READ docs/FESTOS_CONTEXT.md fully. Branch: p3/09-ui-accom. Generate types,
mock with MSW until Prompts 5 and 8 are merged.

BUILD
1. Accommodation workspace: plan generator (choose A/B/C, parameters, budget-
unset banner), side-by-side compare of plans (on-campus, external, unplaced,
contingent splits, total cost, safety flags, accessibility issues, relaxed
constraints) with a cost-versus-cohesion trade-off chart (Recharts) and a
plain-language explanation of what each plan sacrifices.
2. Shortage view: unplaced participants grouped by blocking reason with
counts per contingent, accessibility need and housing status; a Why-not?
button opening a panel with the blocking requirement, missing information,
rejected rooms/properties, and what would change if the field were confirmed,
with a link to verify it.
3. Occupancy map: blocks and floors with beds coloured by status (free,
occupied, cleaning, closed, accessible, unverified), contingent colour
overlay, split highlighting, leader proximity indicator, external properties
with cost and UNVERIFIED accessibility/gender badges.
4. Decision flow: assign/move with live constraint check, violated or relaxed
constraints listed before confirm, ConfirmWithReason, hard-constraint
overrides visible only to the Director, WhyCard on every assignment, override
register link, commit-to-live with approval status tracker.
5. Check-in desk mode: large search/scan field, one-handed layout, shows room
readiness, accessibility and diet alerts, unknown-field prompts with quick
verify, arrival board with pending, arrived and late lists, check-out with
cleaning status.
6. Register a participant-360 'accommodation' tab component for the
participants feature (additive file only; coordinate by exporting from your
feature folder).

UX RULES: nothing colour-only, shortcuts for scan and search, optimistic UI
only where the server is idempotent, concurrency conflicts shown as a clear
message with refresh.
TESTS: vitest for plan compare and constraint list; Playwright: generate
plans, compare, assign with reason, hit a 409 and recover.
ACCEPTANCE: an Accommodation Lead can compare three plans, explain an
unplaced participant, make a justified manual move, and check someone in.
```

Prompt 10 · Person 1 · Inventory ledger, equipment, merchandise QR, duplicate-claim defence

Branch `p1/10-inventory` · **Depends on** 1, 4 · **Unblocks** 12, 14, 21 · **Owns** modules `logistics`, `merchandise`, `models/inventory.py`, `engines/inventory/`, `data/mapping/inventory_*.yaml`

```
READ docs/FESTOS_CONTEXT.md and spec sections 3.10-3.11, 4.4, 5 (Features 5
and 6), 12.3-12.4, 8 (logistics/merch edge cases) fully. Branch: p1/10-
inventory. Push docs/api/logistics.md and merchandise.md first.

GOAL: time- and location-aware inventory where physical stock is never
confused with available stock, and merchandise is issued exactly once to the
right canonical person.

TASKS
1. Ledger: inventory_item (equipment | merch, categories, sizes),
inventory_location, inventory_movement with types RECEIVE, RESERVE, RELEASE,
TRANSFER_OUT, TRANSFER_IN, ISSUE, RETURN, DAMAGE, SEND_REPAIR, REPAIR_DONE,
ADJUST (stocktake, reason mandatory). Never overwrite quantities. A
projection computes on_hand, reserved, in_transit, damaged, under_repair,
awaiting_return, available (document the formula) per item per location and
time; a consistency check proves movements balance. In-transit stock counts
at neither endpoint; transfers carry ETA and an explicit completion step.
Returned merchandise sits in awaiting_return until inspected.
2. Equipment allocation to events/venues with time windows: an item cannot be
at two venues at once; transfer feasibility uses travel_time plus transfer
duration; find_equipment(event_session, window) returns available units,
units that need a transfer (with feasibility) and none. Default one unit when
quantity is unspecified (assumption). Low-stock thresholds and alerts.
Stocktake mismatch flow. The 'camera' case (none in good condition although
three are needed) and the tables 'needs check' inconsistency must surface as
reconciliation flags, not as confirmed shortages.
3. equipment_failure_analysis(item_or_unit) returns dependent sessions
(through schedule public.py), alternative units and transfer feasibility;
used by Prompt 14. Provide the refined EquipmentAvailabilityPort adapter for
Prompt 7.
4. Merchandise: catalog, size management, entitlement policy (assumption: one
per participant record for per-head items; winner medals are not per-head;
policy editable and shown), entitlements generated per CANONICAL participant,
UNKNOWN size flagged, size-wise demand and shortage report clearly marked as
policy-dependent, reservations, counters.
5. Issue flow: POST /merchandise/issue is idempotent and transactional
(SQLite BEGIN IMMEDIATE or equivalent): resolve merged duplicates to the
canonical participant, verify status and entitlement, check stock, insert
merch_claim (unique canonical_participant_id + item_id), write the ISSUE
movement, all or nothing. A second attempt returns 409 ALREADY_ISSUED with
the original timestamp, counter and staff member. Voiding an issue needs a
reason and approval. Issue history includes the volunteer. Volunteer-role
responses expose only minimal fields.
```

```
6. QR: signed HMAC token per participant (and item) using QR_SECRET, SVG/PNG
generation, batch printable sheet, POST /merchandise/scan, and POST
/merchandise/sync for offline batches keyed by client_op_id.
7. Seeders (order 40) for inventory_logistics.csv and inventory_merch.csv via
mapping files. Register participant 360 section 'merchandise'. public.py:
summary (stock by bucket), risks (merch shortages by size, low stock,
unavailable equipment needed in the next window, stocktake mismatches),
dependents (item -> sessions, participants). Report functions for
inventory_plan.csv and equipment_gap.csv.

TESTS (spec section 15): ledger balances; reserved stock never issued twice;
in-transit never counted at both ends; damaged stock unavailable; two
volunteers issuing the same item simultaneously produce exactly one success
(threads); duplicate QR scan yields ALREADY_ISSUED with original details;
merged duplicate participants cannot both claim.
ACCEPTANCE: scan a QR twice and see the second rejected with the first
transaction; move a camera between venues and see availability change by time
and location.
DO NOT build UI.
```

Prompt 11 · Person 2 · Vendors, dietary coverage, shutdown response, notifications, finance

Branch `p2/11-vendors` · **Depends on** 1, 2, 3 · **Unblocks** 12, 14, 15 · **Owns** modules `vendors`, `food`, `finance`, `notifications`, `models/vendors.py`, `engines/vendors/`

```
READ docs/FESTOS_CONTEXT.md and spec sections 3.12, 4.5, 5 (Feature 7), 6.10, 6.14, 6.15, 8 (vendor edge cases) fully. Branch: p2/11-vendors. Push docs/api/vendors.md, food.md, finance.md, notifications.md first.

GOAL: a vendor view that shows real dietary coverage, compliance and settlement, and responds to a shutdown with replacements and drafts.

TASKS
1. Vendors (14): profile, menu, declared dietary categories, rates, contract, payment terms, operating hours, per-meal-slot and per-location capacity (meal_slot entity), status (ACTIVE, SUSPENDED, SHUT_DOWN), assignments to venues/locations/meal slots (vendor_assignment).
2. Compliance: certificate validity vs festival dates computed per day (expired, expires during the festival, valid throughout); a vendor's capacity drops out of the coverage from the expiry moment, not for the whole festival. Menu-label verification: keyword classifier of menu items against declared dietary options; report mismatches (a Jain thali with no Jain declared; vegan bowls with non-vegetarian/halal declared) as flags for human confirmation, never auto-fix. Vendors claiming a category without confirmation show as UNVERIFIED.
3. Demand and coverage matrix: meals needed per meal slot, venue and dietary category from the on-site headcount timeline; UNKNOWN dietary stays its own group and is shown as a demand range, never assumed vegetarian. Matrix categories: vegetarian, non-vegetarian, vegan, Jain, halal, gluten-free, nut allergy, other. Compute valid capacity per slot vs demand, gaps per category, and aggregate vs per-slot shortfall (spec reference: 1,590 valid capacity against ~1,930 on site; verify). Categories with zero compliant vendors are critical alerts.
4. Shutdown response vendor_shutdown_analysis(vendor_id, from_time): the ten steps in spec Feature 7 (affected locations, meal slots, lost capacity, dietary gaps, replacement candidates ranked by dietary suitability, certificate validity, hours, spare capacity, location and cost, backup capacity check, cost change, recommendation, notification drafts, state update after approval via ChangeService handler vendors.shutdown). Backup vendor that cannot absorb the load must be flagged.
5. Notifications (6.14): notification_draft generation per audience (contingent leaders, volunteers, vendors, accommodation coordinators, event organisers) from structured change data; approval required before mock send; delivery log; templates editable. Expose generate_drafts(change_set_or_incident) for Prompt 14.
6. Finance: external accommodation spending (from accommodation public.py), vendor settlements (invoices, payments, due dates from payment terms, disputes, deadlines), merchandise costs, budget utilisation vs
```

```
external_budget_inr, cost variance, approved emergency spending, budget-
override report. Finance Manager role gates all of it; overrides use the
approvals framework.
7. Seeder (order 50). public.py: summary, risks (expired or soon-expiring
certificates, zero-coverage dietary categories, capacity shortfalls, overdue
settlements), dependents (vendor -> meal slots, locations, dietary groups).
Report functions vendor_coverage.csv and vendor_compliance.csv.

TESTS: certificate expiry mid-festival reduces capacity only after the expiry
time; menu mismatch flagged; coverage per slot not just aggregate; UNKNOWN
diet never counted as satisfied; shutdown picks only compliant, valid, open,
spare-capacity replacements; notifications cannot be sent without approval.
ACCEPTANCE: shut down a vendor, see lost meals by slot and dietary gaps,
receive ranked replacements with cost delta, approve, and watch capacity and
drafts update.
DO NOT build UI.
```

Prompt 12 · Person 3 · UI: schedule, logistics, merchandise counter, vendors, finance

Branch `p3/12-ui-ops` · **Depends on** 7, 10, 11 · **Owns** `frontend/src/features/{events,venues,schedule,logistics,merchandise,vendors,food,finance}`

READ docs/FESTOS_CONTEXT.md fully. Branch: p3/12-ui-ops. Generated types plus MSW mocks until backends merge.

BUILD
1. Events & Venues: Gantt timeline by venue with ASSUMED badge on generated schedules, conflict panel grouped by the ten conflict types with severity and affected people, ranked resolution candidates (conflicts resolved, new conflicts, people affected) with apply and approval flow, unscheduled entrants table with reasons and alternatives, judge-gap indicators, schedule version list with diff and restore, event and venue editors including tag editor and compatibility mapping, late-wave and wildcard decisions shown as approval-required.
2. Logistics: stock table showing every bucket (on hand, reserved, in transit, damaged, under repair, awaiting return, available), location view with transfers and ETAs, movement ledger with filters, equipment-to-session assignment with transfer feasibility, low-stock and stocktake mismatch alerts, reconciliation flags for the camera/tables cases.
3. Merchandise admin: catalog, size-wise demand vs stock with a note that shortages depend on the entitlement policy, reservations, issue history with volunteer, QR generator and printable sheet. COUNTER MODE for volunteers: camera QR scan via getUserMedia with manual-ID fallback, huge result panel: green ISSUED, or red ALREADY ISSUED showing the original timestamp, counter and staff member. Volunteers see minimal fields only.
4. Vendors & Food: dietary coverage matrix as a heat grid (category x meal slot x venue) with zero-coverage cells highlighted, compliance table with certificate expiry timeline, menu-label mismatch flags, vendor detail with assignments and rates, shutdown modal showing lost capacity, ranked replacements, cost delta, notification drafts for approval.
5. Finance: budget gauge (budget-unset state explained), external accommodation spend by property, vendor settlements with due dates and disputes, merchandise costs, approved emergency spend, budget-override report.

UX RULES: tables virtualised, keyboard friendly, no colour-only meaning, every alert has a direct action button.
TESTS: vitest for conflict panel, counter result states, coverage grid; Playwright: resolve a conflict, scan a QR twice, shut down a vendor.
ACCEPTANCE: an Event Lead resolves a clash, a volunteer sees ALREADY ISSUED on a rescan, a Food Ops Manager completes a vendor shutdown from the UI.

Prompt 13 · Person 1 · Staff and shifts, duty-of-care and shuttle planner

Branch `p1/13-staff-transport` · **Depends on** 4, 7, 8 · **Unblocks** 14, 15 · **Owns** modules `staff`, `transport`, `models/{staff,transport}.py`, `engines/{staffing,shuttle}/`

```
READ docs/FESTOS_CONTEXT.md and spec sections 3.13-3.14, 5 (Feature 8), 7
(Feature C), 4.3 (volunteer shortage) fully. Branch: p1/13-staff-transport.
Push docs/api/staff.md and transport.md first.

GOAL: know who is where and who covers what, and make sure nobody housed off
campus is left without a safe, verified way home.

PART 1: STAFF AND SHIFTS
1. Staff (48 core team): profiles, roles, skills, availability, shift
schedules (shift table), assignments to events, venues and sessions, handover
notes (a shift change without a handover note is flagged). Seeder (order 60)
from core_team.csv.
2. Volunteer demand: from schedule sessions (volunteers_required) compute
demand over time across simultaneous sessions, compare with staff actually on
shift, list uncovered slots and the maximum gap (spec reference: peak
requirement 74 vs 18 on shift, ~34 hours of gap; verify). Understaffed-event
alerts, overloaded-staff detection (max hours, consecutive hours), shift-
overlap and double-venue detection including travel time. Emergency
replacement suggestions: nearest available, skill match, not overloaded, not
already committed. Flag staff counted as ordinary participants, and staff who
are judges/escorts with a conflict of interest. Assign/replace through
ChangeService handlers staff.assign, staff.replace.

PART 2: DUTY OF CARE AND SHUTTLES
3. Tables and engines for vehicle, route, shuttle_run, shuttle_assignment,
escort_assignment, guardian_approval. All transport parameters come from
assumptions (shuttle_capacity_seats, shuttle_vehicle_count,
shuttle_travel_min, transport_hours, escort availability) and are displayed
as ASSUMED placeholders.
4. Shuttle planning: identify participants in external accommodation (from
accommodation public.py), group journeys by property and time (arrivals,
departures, event commitments), compute required runs = ceil(passengers /
capacity) per window, compare with vehicles/hours, report transport
shortages. transport_feasible(property, time) is exposed for the
accommodation engine's rule 6.
5. Night-safety roster: late-night arrivals/departures (window from
assumptions), minors in external stays, participants without a verified
arrangement; assign named escorts/volunteers (from staff) and track
verification. Guardian-approval queue for minors in external accommodation
(participant, contingent, property, stay duration, approval status, escort,
transport, responsible coordinator; spec reference ~71 minors; verify).
Transport alerts in plain text, e.g. 'N participants leave <property> at
<time> with no verified shuttle'. Safety coverage summary: people needing
```

```
transport, confirmed capacity, uncovered requests, late-night movement,
pending minor approvals, unassigned escorts, property risk.
6. Register participant 360 section 'transport'. public.py for both modules:
summary, risks (uncovered shifts in the next hours, unverified night
transport, minors without approval/escort), dependents (staff ->
sessions/venues; shuttle -> participants). Report function staff_coverage.csv
and resource_load_timeline.csv additions.

TESTS: coverage maths hand-calculated on a small fixture; overlap and
overload detection; emergency replacement never picks an overloaded or
double-booked person; shuttle runs = ceil formula edge cases (exact
multiples, zero passengers); every minor in external stay appears in the
guardian queue; changing an assumption changes the plan.
ACCEPTANCE: the transport dashboard lists people with external stays and no
verified late-night arrangement; staffing shows uncovered hours with
replacement suggestions.
DO NOT build UI.
```

Prompt 14 · Person 2 · Change impact engine, incident and disruption console, replanner

Branch `p2/14-impact` · **Depends on** 4, 7, 8, 10, 11, 13 · **Unblocks** 15, 16, 17, 19 · **Owns** modules `incidents`, `impact`, `engines/impact/`, `engines/replan/`

```
READ docs/FESTOS_CONTEXT.md and spec sections 6.1, 6.6, 12.5, 13, 8 fully.
Branch: p2/14-impact. Push docs/api/incidents.md and impact.md first.

GOAL: the central intelligence layer. Any change or disruption yields its
downstream consequences, ranked recovery plans, human approval, a REAL state
change, and a verification step.

TASKS
1. Dependency graph (engines/impact): built from every module's
public.dependents() plus ORM relations, covering the chains Participant ->
Contingent -> Registration -> Event -> Venue -> Equipment -> Vendor;
Participant -> Arrival -> Check-in -> Accommodation -> Meal -> Vendor;
Participant -> Merchandise eligibility -> Stock -> Distribution counter. BFS
with depth cap and cycle guard; per-node severity and people_affected; output
ImpactReport (tree, counts by entity type, severities, recommendations,
before/after comparison, downstream-update tracking per node PENDING |
UPDATED | VERIFIED). Register the impact hook with core.change_service so any
applied change can show its blast radius. POST /impact/analyze {change}
```

previews any change without applying it.
2. Disruption catalog (GET /disruptions/catalog with parameter schemas). The seven mandatory live-test scenarios, each wired to real module functions:
   - LATE CONTINGENT: update arrivals, run allocate_incremental (Prompt 8), re-check sessions, meals, merchandise entitlements and transport demand.
   - DUPLICATE PARTICIPANT: use the reconciliation merge/undo (Prompt 4), re-point registrations, stay, claims; show affected reservations.
   - HOSTEL CLOSURE: replan_accommodation (Prompt 8) with minimal-change diff; affected participants, cost delta, accessibility/minor safety flags.
   - EQUIPMENT FAILURE: equipment_failure_analysis (Prompt 10); dependent sessions, alternative units with transfer feasibility, reschedule options.
   - VENUE CHANGE: re-run capacity, time, participant, travel and resource checks (Prompt 7); ranked alternatives.
   - VENDOR SHUTDOWN: vendor_shutdown_analysis (Prompt 11); dietary gaps, replacements, cost delta, drafts.
   - DUPLICATE MERCHANDISE CLAIM: demonstrate rejection of the second claim, show the original transaction, flag suspected duplicate identity.
   Extras: bus delayed by N hours, staff shortage, venue unavailable, event cancelled/merged, room unavailable, demand surge, certificate expiry.
3. Lifecycle: incident states OPEN -> ANALYSING -> PLAN_PROPOSED -> APPROVED -> EXECUTING -> VERIFIED -> CLOSED with a timeline. Inject (with dry-run) -> impact preview -> optional scenario simulation (use Prompt 16's sandbox through its public API when available; until then a clear TODO adapter) -> two or more recovery plans compared (people moved, schedule changes, resource moves, cost increase, cohesion) -> approval via approvals framework -> execution through ChangeService (real writes, one change_set per plan) -> verification pass re-running relevant risks and showing before/after -> DecisionCard. Static or hardcoded demos are not acceptable: tests must assert database rows changed.
4. Minimal-change replanner (engines/replan): objective order = fewest people relocated, least schedule disruption, least resource movement, preserve cohesion, least cost increase, preserve approved allocations; report the number of changes versus the original plan.
5. Concurrent disruptions: a second incident arriving while one is mid-resolution is analysed against the post-change state of the first and warned about when pending change_sets touch the same entities (conflict detection between change_sets, queueing and ordering).
6. Notifications: call generate_drafts (Prompt 11) for every plan; approval required to mock-send.
7. API: POST /incidents, GET /incidents, GET /incidents/{id}, GET /incidents/{id}/impact, GET /incidents/{id}/plans, POST /incidents/{id}/approve, POST /incidents/{id}/verify, GET /impact/graph?entity=. public.py for incidents (risks: open incidents, stalled approvals)

```
and the impact module. Report function incident_report.csv.

TESTS: each of the seven scenarios end to end changes real rows and shows
consequences; impact counts equal independently computed expectations on
fixtures; second disruption mid-resolution is detected; replay of an approved
incident is idempotent; rejected plans change nothing; dependency cycles
terminate.
ACCEPTANCE: inject a hostel closure from the API, see the impact tree,
compare two recovery plans, approve one, and verify the live assignments,
audit trail and drafts.
DO NOT build UI.
```

Prompt 15 · Person 3 · UI: disruption console, impact tree, staff, transport, notifications

Branch `p3/15-ui-incidents` · **Depends on** 13, 14 · **Owns** `frontend/src/features/{incidents,impact,staff,transport,notifications}`

```
READ docs/FESTOS_CONTEXT.md fully. Branch: p3/15-ui-incidents. Generated
types plus MSW until backends merge.

BUILD
1. Incident Center / Disruption Console: seven scenario cards (late
contingent, duplicate participant, hostel closure, equipment failure, venue
change, vendor shutdown, duplicate merchandise claim) plus the extras in a
secondary list. Each card opens a guided flow: parameter form -> impact
preview (dry run) -> optional simulate -> compare recovery plans side by side
(people moved, schedule changes, resource moves, cost delta, constraints
relaxed) -> approve with ConfirmWithReason -> execution progress ->
verification screen with before/after and the downstream-update tracker
(PENDING, UPDATED, VERIFIED). Incident list with states and timeline;
concurrent-incident warning banner.
2. Impact Analysis: interactive visual impact tree (expand/collapse by entity
type, severity and people-affected badges, click-through to the entity),
counts by type, before-versus-after comparison, link from any audit entry or
change to its blast radius.
3. Staff & Shifts: roster grid by hour and venue, coverage heat strip (demand
vs on shift) with uncovered slots highlighted, overloaded and overlapping
staff flags, emergency-replace dialog with ranked suggestions, handover notes
editor with 'missing handover' warnings.
4. Transportation & Safety: shuttle demand vs confirmed capacity per property
and hour, ASSUMED parameter banner with edit links, night-safety roster with
escort assignment and verification toggles, guardian-approval queue for
minors, safety coverage tiles, plain-language transport alerts each with an
action.
5. Notifications: drafts inbox grouped by audience (leaders, volunteers,
vendors, coordinators, organisers), edit, approve, mock-send, delivery log.

UX RULES: the live-test path (inject -> approve -> see consequences) must be
completable in under a minute with the keyboard; consequences always visible
on the same screen as the decision.
TESTS: vitest for plan comparison and impact tree; Playwright for the hostel-
closure and vendor-shutdown journeys.
ACCEPTANCE: judges can inject any of the seven disruptions, see the impact
tree, approve a plan, and watch real state change.
```

Prompt 16 · Person 1 · Dashboard backend, attention queue, risk heatmap, decision log, scenario simulator

Branch `p1/16-dashboard-sim` · **Depends on** 14 (and all module public.py files) · **Unblocks** 17, 18, 19 · **Owns** modules `dashboard`, `decisions`, `scenarios`

READ docs/FESTOS_CONTEXT.md and spec sections 5 (Features 9, 11), 6.2, 6.7, 6.8 fully. Branch: p1/16-dashboard-sim. Push docs/api/dashboard.md, decisions.md, scenarios.md first.

TASKS
1. Dashboard metrics: GET /dashboard/metrics aggregates every module's public.summary() into the spec list: total participants, arrived, checked in, pending arrivals, occupancy, available beds, unplaced, active sessions, open conflicts, equipment availability, merchandise stock, vendor status, food coverage gaps, staff availability, open incidents, pending approvals, active overrides. Cache with a short TTL and invalidate on change events. Role-aware subsets (Volunteer gets nothing sensitive).
2. Attention queue: GET /dashboard/attention gathers public.risks() from all modules (iter_public_modules), de-duplicates, ranks by Severity x Urgency x People affected using core.scoring only, includes the example classes in the spec (unconfirmed accessibility needs arriving within six hours, unplaced participants, bed shortages, blocking conflicts in the next four hours, expired certificates, dietary gaps, equipment shortages, understaffed events, pending high-risk approvals, duplicates awaiting review, late/missing arrivals). Every item carries a direct action (route and/or API) to resolve it.
3. Risk heatmap data: GET /dashboard/heatmap returning cells by severity x time-to-impact with people affected, resource criticality and uncertainty, plus the top contributors per cell.
4. Decision log: GET /decisions, GET /decisions/{id}; DecisionCard aggregation across accommodation, schedule, vendors, incidents and overrides, with a one-click 'why' endpoint and filters by module, approver, date. Provide a decision_log.add() helper in core usage docs so modules store cards consistently. Override register view comes from P3's overrides module; link, do not duplicate.
5. Scenario simulator (sandbox): POST /scenarios forks the live DB (SQLite VACUUM INTO or file copy to scenarios/<id>.db; document the Postgres alternative) and binds core.db.use_scenario; apply any disruption/handler, impact analysis and plan generation inside the sandbox; GET /scenarios/{id}/compare returns metric diffs versus live (beds, unplaced, cost, conflicts, coverage); support several scenarios in parallel; POST /scenarios/{id}/commit replays the recorded change list onto live through ChangeService after approval, warning when live changed since the fork (rebase check); POST /scenarios/{id}/discard; scenario history with limits on size and age.
6. Provide the adapter Prompt 14 calls to simulate an incident in a scenario.
7. public.py for dashboard (risks: stale caches, scenarios awaiting decision).

```
TESTS: ranking matches hand-computed scores; each module's risks appear
exactly once; sandbox changes never touch live data (assert row counts and
audit); commit applies exactly the recorded changes; a stale-fork commit is
detected; heatmap bucketing edge cases.
ACCEPTANCE: fork live data, close a hostel in the sandbox, compare, discard,
and show live data untouched; then commit another scenario with approval.
DO NOT build UI.
```

Prompt 17 · Person 2 · Unknowns burn-down, fest pre-mortem, forecasting

Branch `p2/17-foresight` · **Depends on** 5, 14, 16 · **Unblocks** 18 · **Owns** modules `unknowns` (ranking and confirmations), `premortem`, `forecasting`, `engines/{voi,premortem,forecast}/`

```
READ docs/FESTOS_CONTEXT.md and spec sections 7 (Features A and B), 6.4, 6.13
fully. Branch: p2/17-foresight. Push docs/api/unknowns.md (ranking
additions), premortem.md and forecasting.md first.

PART 1: UNKNOWNS BURN-DOWN (value-of-information)
1. Identify unknown field groups (housing preference/gender, accessibility
needs, dietary requirement, arrival/departure time, guardian/escort info,
merchandise size) and affected participants.
2. For each group simulate outcomes by re-running the accommodation engine
and vendor coverage in the scenario sandbox with the field set to each
plausible value (and an explicit distribution where one exists in data;
otherwise report a range, never a point guess). Measure beds made eligible,
unplaced participants reduced, cost change, cohesion gain, dietary coverage
gain, safety uncertainty reduction. Rank by operational improvement per
confirmation. Numbers come from current records and are labelled with their
method.
3. Generate minimal confirmation requests (batched by contingent leader or
desk volunteer, only the field needed), record answers through Prompt 4's
field_verification, re-run the affected logic and show before/after. Maintain
confirmation history. Extend the Why-not explanation (Prompt 5) with 'what
confirming X would change'.

PART 2: FEST PRE-MORTEM
4. Hourly timeline engine across the festival window computing bed occupancy
and availability, food demand vs vendor availability, venue utilisation,
equipment demand, volunteer coverage, transport requirements and critical
risks (call each module's risks(session, now) at each hour; no module code
changes).
5. Disruption generator: targeted (bus delay, hostel closure, vendor stop,
equipment failure, venue unavailable, staff shortage, demand surge, transport
interruption, late contingent) and seeded random multi-disruption runs
executed in scenario sandboxes using the impact engine. Identify single
points of failure (resources whose loss hurts many people with no
alternative), rank risks, suggest preventive actions, keep scenario history.
6. Resilience score: explainable weighted formula over people affected by
simulated disruptions, critical dependencies, accommodation shortage, vendor
redundancy, venue redundancy, equipment availability, staffing coverage,
safety coverage. Document the formula and weights in
docs/RESILIENCE_METHOD.md and in the API response; state clearly that it is a
planning indicator, not a probability of success.

PART 3: FORECASTING
7. Forecast arrival peaks, check-in queues (simple queue model with an
```

```
assumed, editable service rate), meal demand, accommodation demand,
merchandise demand, equipment and volunteer demand from the timetable and
registrations. Label each series PROJECTION (derived from data) or ASSUMPTION
(parameter-driven); never present assumptions as predictions.
8. Report functions for any forecasting outputs; public.py for each module
(risks: top SPOFs, data gaps blocking forecasts).

TESTS: VOI ranking recomputes on changed data; before/after totals reconcile
with a plain engine re-run; pre-mortem timeline equals independent hourly
recomputation on a fixture; resilience score changes in the expected
direction when redundancy is added; seeded randomness is reproducible.
ACCEPTANCE: the API returns a ranked list of unknowns with computed gains, a
scrubbable hourly risk timeline, and a resilience score with its components.
DO NOT build UI.
```

Prompt 18 · Person 3 · UI: overview, simulator, pre-mortem, burn-down, decision log

Branch `p3/18-ui-insight` · **Depends on** 16, 17 · **Owns** `frontend/src/features/{dashboard,scenarios,premortem,forecasting,decisions}` and the unknowns ranking additions

```
READ docs/FESTOS_CONTEXT.md fully. Branch: p3/18-ui-insight. Generated types plus MSW until backends merge.

BUILD
1. Overview (default landing): KPI tiles for every spec metric, each clickable into its module; a prioritised Attention Queue (severity, urgency, people affected, direct action button, snooze/acknowledge with reason), exceptions first with no decorative charts; Risk Heatmap (severity x time-to-impact, bubble size = people, opacity = uncertainty) with drill-down; role-specific variants (Director, Accommodation Lead, Event Lead, Logistics, Food Ops, Finance, Volunteer mobile home).
2. Scenario Simulator: fork live, apply a disruption from the catalog, run impact and plan generation, side-by-side metric comparison against live, parallel scenarios list, commit (approval) or discard, stale-fork warning.
3. Fest Pre-Mortem: timeline scrubber (slider plus play) over the festival window with panels for bed occupancy, food demand vs vendor availability, venue utilisation, equipment demand, volunteer coverage, transport, and critical risks appearing as the slider moves; disruption runner, SPOF list, preventive actions, resilience score with a component breakdown and the methodology link; scenario history.
4. Unknowns Burn-down: ranked list of unknown fields with computed gains and method label, minimal confirmation request generator, record-answer form, before/after panel, confirmation history. Add a Why-not? deep link from unplaced participants.
5. Forecasting charts (Recharts) with PROJECTION vs ASSUMPTION legend and editable assumption links.
6. Decision log: list with filters and a Why card drawer, override register link, approver and cost impact columns.

UX RULES: the Overview must be useful in under five seconds of looking; every alert has an action; avoid chart junk.
TESTS: vitest for ranking, heatmap and scrubber state; Playwright: open attention item -> land on the fixing screen; scrub the timeline; fork and discard a scenario.
ACCEPTANCE: dragging the scrubber to a critical hour reveals risks in advance; a Director can fork a hostel-closure scenario, compare, and discard without touching live data.
```

Prompt 19 · Person 1 · AI command center and reports/exports

Branch `p1/19-commands-reports` · **Depends on** 14, 16 · **Unblocks** 21 · **Owns** modules `commands`, `reports`, `reports/`, `engines/commands/`

```
READ docs/FESTOS_CONTEXT.md and spec sections 6.12, 6.16, 14 fully. Branch:
p1/19-commands-reports. Push docs/api/commands.md and reports.md first.

PART 1: NATURAL-LANGUAGE COMMAND BAR (AI proposes, deterministic logic
validates, humans approve)
1. Define a strict structured-command schema (intent enum plus typed
parameters): show_impact(entity), find_accommodation_for_unplaced,
affected_events_by_equipment(item), alternative_vendors_for_diet(diet),
list_late_arrivals(after_time), plus generic query and navigate intents. Add
intents only by adding schema entries.
2. LLM adapter (provider-agnostic through LLM_PROVIDER and LLM_API_KEY) that
converts free text to that schema ONLY. Ship a deterministic offline fallback
(keyword and fuzzy intent matcher) so the feature works with no key and
during a network outage. Treat all retrieved data as data, never as
instructions; the model gets no tools and no database access; send only the
minimum non-sensitive context (no contact, medical or accessibility details).
3. Pipeline: POST /commands/parse -> structured command plus an 'I
interpreted this as...' explanation; POST /commands/execute: the
deterministic validator checks existence, parameters and the caller's
permissions; read-only intents run immediately; any mutating intent only
creates a PROPOSAL (ChangeService dry_run + impact preview) that must go
through the approvals framework. Log every command and AI interpretation to
the audit trail. Rate limit the endpoint.

PART 2: REPORTS AND EXPORTS
4. A registry in reports/ with one generator per file in spec section 14:
reconciliation_report.csv, data_quality_report.csv,
accommodation_plan_A/B/C.csv, shortage_report.csv, accommodation_summary.csv,
schedule_plan.csv, conflicts_report.csv, unscheduled_entrants.csv,
inventory_plan.csv, equipment_gap.csv, vendor_coverage.csv,
vendor_compliance.csv, staff_coverage.csv, bed_demand_timeline.csv,
resource_load_timeline.csv, assumptions.md, incident_report.csv,
audit_report.csv. Reuse the report functions earlier modules exposed; add the
missing ones.
5. Endpoints: GET /reports (catalog with required permission), GET
/reports/{name}, GET /reports/bundle.zip. Exports obey masking and export
restrictions per role, neutralise CSV injection (prefix cells starting with
=, +, -, @), and every export is audited.
6. Post-festival report generated from audit history and live state:
participant attendance, accommodation utilisation, event completion,
inventory usage, vendor performance, incident history, financial expenditure,
operational delays, outstanding settlements. HTML and CSV outputs.
7. Produce the sample plans for deliverable 02 into data/golden/ with a
```

```
script `make samples` (accommodation A/B/C, schedule, inventory) and golden-output regression tests.

TESTS: parser maps each example command from the spec to the right schema entry (offline fallback included); prompt-injection strings inside participant names never alter a command; a mutating command never executes without approval; reports respect masking per role; CSV injection neutralised; golden outputs stable across `make reset`.
ACCEPTANCE: 'Show the impact of closing Hostel C' returns an impact report; 'Find accommodation for the unplaced participants' creates a proposal needing approval; all 20 outputs download.
DO NOT build UI.
```

Prompt 20 · Person 2 · Hardening: full test matrix, concurrency, security review

Branch `p2/20-hardening` · **Depends on** 1 to 19 · **Unblocks** 22, 23 · **Owns** `backend/tests/`, `backend/tests/e2e_api/`; cross-owner fixes go through small labelled PRs

```
READ docs/FESTOS_CONTEXT.md and spec sections 15, 8, 20 fully. Branch: p2/20-hardening. Use label cross-owner for any fix outside your folders and request review from the owner.

GOAL: prove it works and survives abuse. Treat every module as untrusted.

TASKS
1. Build the test matrix from spec section 15 and commit docs/TEST_MATRIX.md mapping every bullet to a test file and status:
   - Accommodation: accessibility never violated, no capacity exceeded, no bed double-booked, cleaning respected, UNKNOWN housing never silently assigned, every unplaced participant has a reason.
   - Scheduling: no venue double-booking, no overlapping shared participants, no insufficient travel windows, no equipment double allocation, no volunteer double assignment, nothing outside venue hours.
   - Inventory: movements balance, reserved stock never issued twice, in-transit not counted twice, damaged unavailable, duplicate merchandise claims yield one success.
   - Data: malformed CSV, missing columns, quarantine, UNKNOWN stays UNKNOWN, merges reversible, raw records preserved.
   - System: RBAC enforced, sensitive data masked, changes audited, concurrent updates use version checks, failed transactions leave no partial state, seed reset works, golden tests reproduce outputs.
2. Property-based tests (hypothesis) on the allocation and scheduling engines
```

```
with randomised contingents, accessibility needs, UNKNOWN genders, minors and
late arrivals.
3. Concurrency tests with real threads or processes: same bed allocated
twice, same merchandise issued twice, same record edited twice, two
disruptions touching the same entity, simultaneous check-in sync replays.
Fault injection: raise inside handlers mid-transaction and assert no partial
rows, no audit gaps.
4. Determinism: `make reset` twice gives identical golden outputs; seeded
randomness reproducible; time-dependent code uses core.clock only (search for
datetime.now / utcnow and replace with a PR).
5. Security review checklist with fixes: authorisation on every endpoint
(IDOR checks by role and ownership), JWT secret strength and expiry, CORS,
login rate limiting, upload size/type limits and zip bombs, SQL injection
through filters and sort params, PII in logs and error messages, QR token
forgery and replay, break-glass scoping, export masking, CSV injection,
prompt injection in the command bar, secrets in the repo.
6. Performance smoke: list endpoints, dashboard and plan generation at 2,836
participants with p95 targets recorded; add indexes where needed.
7. Validate the validators: deliberately corrupt the data pack (missing
columns, invalid dates, duplicate IDs) and confirm clear errors.
8. Publish docs/TEST_REPORT.md with pass/fail, known gaps and their severity;
open issues for anything you could not fix.

ACCEPTANCE: `make test` is green and fast enough for CI; the matrix has no
unexplained gaps; each critical bug found has a fix PR or a documented
limitation.
DO NOT add features.
```

Prompt 21 · Person 3 · PWA offline scanning, volunteer view, command bar and reports UI, polish

Branch `p3/21-pwa-polish` · **Depends on** 8, 10, 19 · **Owns** `frontend/src/pwa/`, `frontend/src/features/{commands,reports}` and cross-app polish

```
READ docs/FESTOS_CONTEXT.md and spec sections 6.9, 6.11, 6.12 fully. Branch:
p3/21-pwa-polish.

BUILD
1. PWA (vite-plugin-pwa): installable, app shell cached, runtime caching for
read-only lookups with a visible 'last synced' stamp. Offline queue in
IndexedDB (Dexie) for ONLY safe operations: check-in and merchandise
scan/issue. Each queued op has a client_op_id (UUID), timestamp, device id,
minimal payload; never queue overrides, allocations, merges or approvals.
Online/offline indicator, pending-count badge, automatic sync on reconnect
plus manual sync, calling /checkin/sync and /merchandise/sync. Show per-op
results (APPLIED, DUPLICATE_IGNORED, CONFLICT) with the server's explanation;
conflicts need a human decision screen. Cached lookup data is minimal and
encrypted at rest where the browser allows, wiped on logout, and stale data
shows a warning.
2. Volunteer mobile view: when the role is Volunteer the app shows only two
big flows (Check-in, Merchandise counter) with large buttons, high contrast,
haptic or sound feedback, almost no personal data. Wire the counter mode from
Prompt 12 to the offline queue.
3. AI Command Center UI: global command bar (press Ctrl/Cmd+K) with
suggestions from the spec examples, an 'I interpreted this as...'
confirmation card with editable parameters, results rendering, and proposals
routed into the approvals inbox. Works with the offline fallback parser.
4. Reports page: catalog of all 20 outputs with permission badges, preview,
download, bundle zip, post-festival report preview, assumptions and
limitations viewer.
5. Usability under pressure: consistent loading, error and empty states,
toast patterns, undo where the backend supports it, skeleton screens,
keyboard shortcuts documented in a help overlay, WCAG AA contrast, aria
labels, focus order, responsive down to 360 px, virtualised long lists,
bundle-size and route-level code splitting.

TESTS: Playwright with network offline/online toggling: queue three scans
offline, reconnect, verify exactly-once application and a duplicate being
ignored; role-gated navigation for Volunteer; command bar flow; vitest for
the queue and conflict screen.
ACCEPTANCE: a volunteer on a phone can check people in and issue merchandise
while the Wi-Fi drops, and on reconnect nothing is applied twice.
```

Prompt 22 · Person 1 · Final integration, twist playbook, Docker deployment, code freeze

Branch `p1/22-integration` · **Depends on** 1 to 21 · **Unblocks** 23, 24 · **Owns** repo root, CI, Docker, `scripts/`, `docs/TWIST_PLAYBOOK.md`

```
READ docs/FESTOS_CONTEXT.md, docs/INTEGRATION_LOG.md and docs/TEST_REPORT.md
fully. Branch: p1/22-integration.

GOAL: one coherent, fast, resettable product on main, with a documented way
to absorb a surprise.

TASKS
1. Merge train: review open PRs into develop in dependency order, resolve
conflicts by consulting owners, run the full CI, then merge develop to main.
Regenerate API types and fail CI on contract drift.
2. scripts/smoke_all.py: logs in as each role, hits every module's list
endpoint, runs the seven disruption scenarios through the API, asserts that
state changed and the audit trail grew, and prints a readable summary. Fix
integration bugs found, or hand them to owners with reproduction steps.
3. Cross-module consistency checks in one script: every public.py exposes
summary/risks/dependents; every mutation route uses ChangeService; every
module slug has a nav entry and docs/api file; no unmasked sensitive fields
in any serializer.
4. Deployment: production-style docker-compose (backend, frontend served by
nginx, volume for the SQLite file, healthchecks, restart policy, non-root
users, env from .env), `make demo` one command from clean clone to running
with seeded data and demo users, backup/restore commands,
data/snapshots/demo.db as a known-good fallback, a `make reset-demo` that
restores it, pinned dependencies and lockfiles.
5. Feature flags in config to switch off any unstable optional module
(premortem, commands, forecasting) without breaking navigation.
6. docs/TWIST_PLAYBOOK.md: how to add a new hard constraint, a new disruption
type, a new report, a new assumption key, a new participant attribute, a new
role; where each lives; what tests to add. This is how the team absorbs the
hour-12 twist and the live-test.
7. Tag releases: v1.0-rc1 at code freeze, v1.0 final; protect main.

ACCEPTANCE: from a clean machine, `git clone && cp .env.example .env && make
demo` yields a working portal in a few minutes; smoke_all passes; main equals
the tested build.
DO NOT add new features after the code-freeze hour.
```

Prompt 23 · Person 2 · Submission pack: README, setup, credentials, AI disclosure, sample plans

Branch `p2/23-submission` · **Depends on** 20, 22 · **Owns** `README.md`, `docs/*` submission files, `submission/`

```
READ docs/FESTOS_CONTEXT.md, spec section 17 and the problem statement slide
on rules and deliverables. Branch: p2/23-submission.

GOAL: every required deliverable present, accurate and verifiable.

TASKS
1. README.md: what FestOS is, architecture diagram (Mermaid), module map,
setup instructions for local run and Docker, test credentials table (from
docs/TEST_CREDENTIALS.md), how to reset the demo, how to run tests, folder
guide.
2. docs/ASSUMPTIONS.md generated from the live assumption table (name, value,
source, owner, reason, impact) and docs/LIMITATIONS.md covering at least:
greedy allocation is heuristic and not provably optimal, travel and shuttle
values are assumed, no real transport or notification integrations, vendor
and accessibility data needs manual confirmation, data is synthetic, SQLite
concurrency limits, generated schedule quality is limited by missing times,
venues, judges and team IDs, the 2,836 vs 2,400 vs 2,842 reconciliation
outcome.
3. docs/AI_DISCLOSURE.md and the content for the single AI slide in three
columns: what AI did (scaffolding, boilerplate, tests, data analysis,
duplicate suggestions, command interpretation, drafts), what the team decided
(constraint hierarchy, accessibility/privacy rules, UNKNOWN treatment, safety
requirements, override policy, human approval), how outputs were checked
(unit and property tests, hand-calculated examples, constraint validation,
plan review, manual review of flagged records). Be specific and honest.
4. submission/sample_plans/: the sample accommodation (A/B/C), scheduling and
inventory plans generated from the pack with the exact command and commit
hash used, plus the reconciliation and data-quality reports.
5. scripts/verify_claims.py: reads every number quoted in README, LIMITATIONS
and the pitch notes (participants, beds, conflicts, unplaced, etc.) and fails
if it differs from the live system; run it in CI.
6. docs/PRIVACY_AND_SECURITY.md: roles, masking, break-glass, audit chain,
export controls, what is not protected and why.
7. docs/API.md index generated from OpenAPI.

ACCEPTANCE: a reviewer who has never seen the project can set it up from the
README, log in with the documented credentials, and find every deliverable;
verify_claims passes.
```

Prompt 24 · Person 3 · End-to-end QA, live-test rehearsal, demo script and pitch

Branch `p3/24-qa-pitch` · **Depends on** 21, 22 · **Owns** `frontend/e2e/`, `docs/DEMO_SCRIPT.md`, `docs/PITCH.md`, `submission/deck/`

```
READ docs/FESTOS_CONTEXT.md and spec sections 13 and 17 fully. Branch: p3/24-qa-pitch.

TASKS
1. Playwright end-to-end suite on the demo build (make demo): login per role; the seven live-test scenarios each asserted to CHANGE REAL STATE and show consequences on screen (late contingent, duplicate participant, hostel closure, equipment failure, venue change, vendor shutdown, duplicate merchandise claim); override requires a reason; break-glass leaves an audit entry; offline scanning exactly-once; scenario fork leaves live untouched. Run on desktop and a 360 px mobile viewport; run an accessibility scan (axe) and fix serious issues.
2. Rehearsal harness: `make reset-demo` returns to a known state in under a minute; a checklist for pre-demo health checks; timing of each disruption flow.
3. docs/DEMO_SCRIPT.md following the seven-minute structure: 0:00-0:45 problem; 0:45-1:45 data reconciliation and real operational problems; 1:45-3:00 accommodation plans A/B/C and trade-offs; 3:00-4:30 live hostel disruption and impact analysis; 4:30-5:30 scheduling, inventory and vendors; 5:30-6:15 security, permissions, audit; 6:15-6:45 assumptions and limitations; 6:45-7:00 AI disclosure. Include exact clicks, expected on-screen results and which team member speaks.
4. docs/PITCH.md: slide outline (clear choices, evidence, limitations), plus three-minute Q&A prep: likely judge questions (why not exactly 2,400, why heuristic not optimal, how unknown data is handled, how privacy is enforced, what happens if two organisers collide, what the AI actually did, what you would build next) with honest, specific answers, and a plan for judges injecting an unplanned disruption.
5. Build or outline the deck in submission/deck/ and a screen-recorded fallback video of the main flows.
6. Final go/no-go checklist at the code-freeze hour: CI green, tags set, demo DB snapshot verified, credentials work, README setup tested on a second machine, laptops charged, offline fallback ready.

ACCEPTANCE: the full demo runs twice in a row in under seven minutes with no manual database edits, and every claim made in the script is visible on screen or in the repo.
```

D. Coverage check: where every spec requirement lives

Spec item	Prompts
Unified participant record, search, status, cancellations, no-shows	4, 6
Contingent management, fragmentation, leader/escort	4, 6
Import, validation, quarantine, UNKNOWN as first-class, assumptions register	2, 4, 6, 17
Duplicate reconciliation workbench, merge/undo	4, 6
Accommodation hard rules 1 to 8, plans A/B/C, explainability, shortages, Why-not	5, 8, 9
Overrides, approvals, override register, break-glass	3, 8, 9
Event/venue scheduling, ten conflict types, resolutions, versions, ASSUMED labelling	7, 12
Logistics ledger, location-aware equipment, transfers	10, 12
Merchandise QR, duplicate claim defence, idempotency	10, 12, 21
Vendors, certificates, dietary matrix, menu check, shutdown and fallback, notifications	11, 12, 14
Finance, settlements, budget, post-festival report	11, 12, 19
Staff and shifts, volunteer demand, replacements, handover	13, 15
Duty-of-care, shuttle planner, night-safety, guardian approvals	13, 15
Change impact engine, incident centre, minimal-change replanner, seven live scenarios	8, 14, 15, 22, 24
Dashboard, attention queue (Severity x Urgency x People), risk heatmap	16, 18
Decision log and Why cards	5, 8, 16, 18

Spec item	Prompts
Scenario simulator	16, 18
Fest pre-mortem, resilience score, timeline scrubber	17, 18
Unknowns burn-down, Why-not counterfactuals	5, 17, 18
Forecasting	17, 18
Natural-language command bar	19, 21
Offline PWA check-in and scanning, role-aware volunteer UI	8, 10, 21
RBAC, masking, audit chain, export restrictions	3, 19, 20
All 20 reports and exports	5, 7, 10, 11, 13, 14, 19
Testing and reliability (spec section 15)	every prompt plus 20
Edge-case lists (spec section 8)	4, 7, 8, 10, 11, 13, 14, 20
Deliverables, setup, credentials, AI slide, pitch, limitations	22, 23, 24

E. Coordination toolkit

How collisions are avoided

- Each person edits only their own folders (Section A.3). Routers, feature folders, seeders, change handlers, participant sections and generated types are all auto-discovered, so adding a module never edits a shared file.
- Modules talk only through `public.py`, `schemas/common.py`, the bus, and ChangeService.
- The contract (`docs/api/<module>.md` plus schema stubs) is pushed before implementation, so the frontend can mock and rebase later.
- Additive schema changes only after round 1; announce them; `make reset` rebuilds.

Checkpoints (merge develop to main)

1. After round 1: skeleton runs, three roles can log in, import loads the pack.

2. After round 3: plans, schedule, check-in work end to end with UI.
3. After round 5: all seven disruption scenarios change real state.
4. After round 7 (code freeze): everything merged, hardening report done.
5. After round 8: tagged release, rehearsed demo.

Twist Prompt (run at the hour-12 twist, by whoever owns the affected area; the other two stay out of those files)

```
READ docs/FESTOS_CONTEXT.md and docs/TWIST_PLAYBOOK.md (if it does not exist
yet, use sections 4, 7, 8 of the context file). A twist has been announced:
<PASTE THE TWIST>.
1. Classify it: new hard constraint, new soft preference, new data field, new
disruption type, changed capacity/quantity, changed policy, or new report.
2. List the modules affected (use public.dependents and the impact engine)
and the people who own them. Do not edit files you do not own; request
changes through a labelled PR.
3. Implement it the smallest safe way: an assumption key, a handler, a risk
provider, or a new rule in the relevant engine, always through ChangeService
with audit, Why cards and visible consequences.
4. Add tests that fail before and pass after, regenerate API types, update
docs/ASSUMPTIONS and the assumptions register, add an INTEGRATION_LOG entry.
5. Run scripts/smoke_all.py and report what changed in real system state.
```

If you are running late, cut in this order (never cut RBAC/audit or the seven live scenarios): forecasting, pre-mortem, natural-language command bar, finance extras, OR-Tools solvers, PWA polish.